Auditoria técnica

NES N8 Everdrive Covers editor

Firmware v15 com capas de jogos para o EverDrive N8 clone (EDFC) — documento para desenvolvedores

TavinhoGames · 30/10/2026

`OS.BIN` v15 SHA-1 `caa12ed797d374341a40a76a7d2124ee3ce51e8a` · módulo `MOD_v10.BIN` SHA-1 `6e5e9cbfe33f9ff11185bb6a5e818528b4595650`

Sumário

1. Resumo 4

2. Linha do tempo das versões 4

3. O hardware e o OS original 5

3.1 Layout do OS.BIN (57.344 bytes) . . . 5

3.2 Registradores do cartucho . . . 5

3.3 Mapa de memória em tempo de execução . . . 5

3.4 Vídeo (PPU) no OS original . . . 5

4. Ponteiros do OS base (build do clone, SHA-1 dbea89d3…) 6

4.1 Rotinas . . . 6

4.2 Variáveis e dados . . . 6

5. Arquitetura da firmware v15 7

5.1 Mapa completo dos patches (diferença byte a byte v15 × original) . . . 7

5.2 Segmentos residentes e espaço livre . . . 7

5.3 Endereços principais do código novo (v15) . . . 8

5.4 Fluxo de execução . . . 8

5.5 Técnicas de vídeo (por que funciona) . . . 9

6. Formato do COVERS.BIN 9

7. Os patches das últimas versões em detalhe 9

7.1 Texto @tavinho.games (v13) . . . 9

7.2 20 jogos por página (v13) — 13 operandos trocados no lugar . . . 9

7.3 ← → no topo da página (v14) . . . 10

7.4 Correção do travamento após ligar (v15) . . . 10

8. Conversor de imagens (PC) 10

9. A ferramenta (ferramentas/gerador_firmware_capas.py) 10

10. Testes (no PC, antes do aparelho) 11

11. Bugs encontrados, causas e lições 11

12. Limites e quirks 12

13. Aplicações e ferramentas usadas 12

14. Como reconstruir tudo 12

Apêndice A — Tabela de símbolos da firmware v15 14

Constantes e equates . . . 15

Apêndice B — Notas técnicas completas (NOTAS-TECNICAS-FIRMWARE.md) 17

0. Ferramentas e como a análise foi feita . . . 17

1. Layout do arquivo OS.BIN . . . 17

2. Hardware: registradores do cartucho . . . 18

3. Mapa de memória em tempo de execução . . . 18

4. Vídeo (PPU) . . . 19

5. Navegador de arquivos . . . 19

6. Projeto proposto para as capas . . . 20

7. Testes pendentes no hardware (diagnóstico) . . . 21

8. Mapa de funções (resumo) . . . 21
9. Pacote completo da firmware (nesos-v1.26) . . . 21
10. Build BASE do clone (o que vale para a custom firmware) . . . 22
11. Resultado do diagnóstico no hardware (DIAG 1, foto em imagens/1.jpeg) . . . 22
12. Firmware com capas do SD (capas_sd/, ferramentas/build_capas.py) . . . 23
14. EXPERIMENTO v2: sprites + camada de fundo (experimento_v2/, ferramentas/build_capas_v2.py) . . . 23
16. Palette Lab local (ferramentas/palette_lab/) . . . 24
17. POC v3: alternância de quadros (experimento_v3/, ferramentas/build_capas_v3.py, conv_v3.py) . . . 25
18. TESTE 80x96 (25/10/2026) — ferramentas_80x96/, Gerador 80x96 (TESTE).bat, experimento_80x96/ . . . 28
19. Investigação: bancos de CHR com o menu rodando (25/10/2026) . . . 28
20. POC: texto fixo no navegador (26/10/2026) . . . 29
21. Associação inteligente ROM ↔ capa (26/10/2026) . . . 29
22. POC: capa em tela cheia (26/10/2026) . . . 29
15. Refazer a análise . . . 30
23. POC v4: capa grande no navegador + tela cheia (26-27/10/2026) — FUNCIONOU no aparelho . . . 30
24. POC v5: + sprites por cima da capa grande (27/10/2026) . . . 30
25. POC v6: a v5 carregando mais rápido (27/10/2026) — a v5 fica como estável . . . 31
26. POC v7: vblank estendido na carga (27/10/2026) — a v6 fica como estável . . . 31
27. POC v8: flicker opcional nos sprites, por capa (27/10/2026) — a v7 fica como estável (funcionou) . . . 31
28. POC v9: cor livre da paleta 3 (27/10/2026) . . . 32
29. POC v10: espera antes de carregar + cancelamento (27/10/2026) — a v9 funcionou no aparelho . . . 32
30. A v10 na ferramenta (27/10/2026) . . . 32
31. POC v11: "@tavinho.games" embaixo da capa (29/10/2026) — a v10 fica como estável . . . 33
32. POC v12: 20 jogos por página (29/10/2026) — a v11 fica como estável . . . 33
33. POC v12b: sem o "lixo" ao ligar (29/10/2026) . . . 34
34. v12 trava ao abrir pastas; POC v13 = v10 + texto + 20 por página (30/10/2026) . . . 34
35. A v13 na ferramenta (30/10/2026) . . . 34
36. A ferramenta em .exe (30/10/2026) . . . 35
37. POC v14: ← → levam o cursor para o topo da página (30/10/2026) . . . 35
38. POC v15: o travamento ao carregar capas depois de ligar (30/10/2026) . . . 35
39. v15 na ferramenta = VERSÃO ESTÁVEL (30/10/2026) . . . 35

Documento para **desenvolvedores**: o que foi feito, o que foi descoberto no OS do cartucho, os endereços (ponteiros), os patches, o formato dos arquivos, os algoritmos, os testes, os bugs e as ferramentas usadas. Data: **30/10/2026**. Autor do projeto: **TavinhoGames** (análise e código feitos com o Claude Code).

Item	Valor
Versão estável	firmware **v15** + ferramenta "NES N8 Covers Editor" (Python / .exe)
`OS.BIN` v15 (com a espera padrão de 0,5 s)	SHA-1 `caa12ed797d374341a40a76a7d2124ee3ce51e8a`
Módulo das capas (`MOD_v10.BIN`, 1 KB)	SHA-1 `6e5e9cbfe33f9ff11185bb6a5e818528b4595650`
OS original do clone (base de todos os patches)	SHA-1 `dbea89d33eacd7ae72ee24d2652727b0cb5740ae` (57.344 bytes; **não distribuído**)
Hardware	EverDrive N8 **clone** (pasta `EDFC`) num NES/Famicom NTSC
Testado no aparelho	817 capas, várias pastas, liga/desliga repetido, navegação, cancelamento

Legenda de confiança usada nas notas: **[OBS]** visto no código/bytes · **[INF]** inferido com boa evidência · **[?]** hipótese.

1. Resumo

O cartucho roda um "OS" de 6502 (escrito em C e compilado com **cc65**) carregado de `/EDFC/OS.BIN`. O projeto modifica esse arquivo (patches no lugar + código novo em áreas recuperadas) para mostrar, no navegador de arquivos, a **capa do jogo selecionado** (112×160 pontos) ao lado da lista. As capas ficam num pacote `/EDFC/COVERS.BIN` gerado no PC por uma ferramenta em Python que converte PNGs para o formato do NES.

Pontos principais da solução:

- **Capa = fundo + sprites.** O fundo usa 2 paletas próprias por capa (bloco de 16×16) e tiles em **7 bancos de CHR trocados no meio do quadro** (split por sprite 0). Por cima, até **62 sprites 8×16** com 4 paletas, e **flicker** opcional (2 quadros alternados a 60 Hz) para mais cores.
- **Código em duas partes:** ~1,2 KB **residente** dentro do OS.BIN (em código morto e nas telas "Device Info" e "About", removidas) + um **módulo de 1 KB** que fica no `COVERS.BIN` (setores 257–258) e é carregado na WRAM `$6400–$67FF` quando preciso.
- **Carga em ~45–65 quadros** (~1 s): a tela é desligada só abaixo da barra de status, numa "janela" de vblank estendido (linha 226 até o pré-render), sincronizada pelo sprite 0.
- **Espera configurável** antes de carregar (o cursor parado em cima do jogo) e **cancelamento** por qualquer botão.
- **20 jogos por página**, ← → levam o cursor para o topo da página nova, `@tavinho.games` embaixo da capa.
- Tudo verificado por **testes no PC** com um emulador 6502 próprio e um **modelo de tempo da PPU** (`ppu_tempo.py`), antes de ir para o aparelho.

2. Linha do tempo das versões

Versão	O que mudou	`OS.BIN` (SHA-1)	No aparelho
PoC base	capa fixa 64×96 em sprites, gravada no próprio OS.BIN	—	funcionou
v1 (`capas_sd`)	capas 64×96 lidas do SD (`COVERS.BIN` v1, índice por CRC)	`7c9f4886…`	funcionou (favorita da época)
v2	sprites + camada de fundo	`c0a74aea…`	funcionou
v3	alternância de quadros (capa pequena)	`5b23f07e…`	funcionou
v4	**capa grande 112×160** só de fundo, split de bancos de CHR	—	funcionou
v5	+ até 62 sprites por cima; módulo de 1 KB em `$6400`	`79894e96…`	lixo ao trocar de capa → 2 pares por vblank
v6	carga mais rápida (`vbw` pelo flag `$9E` do NMI do OS)	—	ok
v7	**vblank estendido** (janela pelo sprite 0); marca do módulo = SHA-1	—	funcionou (depois das correções)
v8	flicker opcional nos sprites, por capa	—	funcionou
v9	cor livre na paleta 3	—	funcionou
v10	espera + cancelamento; atributos da capa fora do buffer do OS	`4c7281b9…`	**funcionou** (estável até 30/10)
v11	`@tavinho.games` + mudança no módulo (`$05`)	`dfede28f…`	ok, **abandonada** (mudança no módulo era desnecessária)
v12	v11 + 20 por página	`768f9cc1…`	travava ao abrir pastas (causa real: ver v15)
v12b	tentativa de tirar o "lixo" ao ligar	`6c27725b…`	capas bugaram → **abandonada**

Versão	O que mudou	OS.BIN (SHA-1)	No aparelho
v13	v10 + texto + 20 por página (sem mexer no módulo)	96d89601…	funcionou
v14	← → levam o cursor ao topo da página	876d21bf…	—
v15	v14 + **correção do travamento após ligar**	caa12ed7…	**funcionou — ESTÁVEL**

Os builders são **incrementais**: `build_capas_v15.py` importa a v14, que importa a v13, … até `build_poc.py`, que lê o OS original. Cada versão só descreve a diferença para a anterior.

3. O hardware e o OS original

3.1 Layout do `OS.BIN` (57.344 bytes)

Offset no arquivo	Conteúdo	Destino
`0x0000–0x0FFF`	CHR tabela 0: duas cópias da fonte (`$00–$7F` normal, `$80–$FF` destaque)	PPU `$0000`
`0x1000–0x1FFF`	CHR tabela 1 (sem uso pelo menu original)	PPU `$1000`
`0x2000–0x5FFF`	preenchimento (`$AA` / `$00`), **não é carregado** pela BIOS	—
`0x6000–0xDFFF`	PRG (32 KB, código 6502 do cc65)	CPU `$8000–$FFFF`

Conversão: `offset = endereço_CPU − 0x8000 + 0x6000` (`build_poc.cpu2file`). Vetores: NMI `$80B4`, RESET `$8000`, IRQ `$80EB`. O PRG roda **em RAM**: pode ser patchado e ter variáveis dentro de `$8000–$FFFF`.

3.2 Registradores do cartucho

Banco indexado: escreve-se o índice em `$4400` e o valor em `$4401`.

Índice	Uso
`$09`	o que aparece em `$8000–$FFFF` (RAM do OS / flash da BIOS)
`$80`	banco de PRG ao carregar o jogo
`$81`	`$08` = WRAM normal em `$6000`; `$09` = mapeia o banco que contém o CHR do OS.BIN
`$82`	**banco de CHR**: janela de 8 KB começando em valor × 4 KB (usado pela capa para trocar de banco no meio do quadro)
`$83`, `$86`, `$88`, `$8B`, `$8C`	configuração do jogo (mapper, mirroring, save)

> Não mexer nos stubs de flash `$847D`, `$84C0`, `$84F3`: eles mapeiam a flash da BIOS e enviam comandos de apagamento — risco de brick. O OS base tem também uma verificação anti-clone em `$B255` ("ERROR 5FA1.Clone detected!") que **não** foi tocada.

3.3 Mapa de memória em tempo de execução

Faixa	Uso
`$0000–$00FF`	zero page. cc65: `c_sp=$28/$29`, `sreg=$2A`, `ptr1..ptr4=$30..$37`, `tmp1=$38`. OS: `$92/$93` ponteiro de texto, `$9E` flag do NMI, `$9F` cor do texto, `$A0` joypad, `$A2` NMI ocupado. A BIOS usa `$80–$8A`
`$0100–$01FF`	pilha do 6502
`$0200–$02FF`	**sem uso no OS** → página de OAM dos sprites da capa
`$0300–$049A`	DATA do cc65 (copiado de `$FA39` no boot)
`$049B–…`	BSS; tabela da API da BIOS copiada de `$FC00`
`$0600–$07FF`	pilha de software do C (cresce para baixo a partir de `$0800`)
`$6000–$63FF`	**buffer da tela** (espelho da nametable `$2000`; atributos em `$63C0`)
`$6400–$67FF`	**módulo das capas** (WRAM com **bateria**!). O OS usa `$6400–$66FF` como rascunho no boot
`$6800–$7EFF`	heap do OS (tabelas do diretório, buffers de setor)
`$7F00–$7FFF`	variáveis da capa (atributos `ATC`, contadores, cópia dos metadados dos sprites)
`$FBA2–$FBFF` (build base: `$FBD4–$FBFF`)	livre no arquivo
`$FC00–$FFF9`	**preenchido pela BIOS** em tempo de execução (API, driver de SD) — NÃO está livre

3.4 Vídeo (PPU) no OS original

- `PPUCTRL=$80`, `PPUMASK=$0A` (fundo ligado, **sprites desligados**: o OS nunca usa sprites).

- Paleta do fundo em `$825F`: `0F 10 0F 28 | 0F 0F 0F 0F | 0F 0F 0F 0F | 0F 30 00 28`. Paleta 0 = lista (cor 3 = amarelo de pastas/cursor), paleta 3 = barra (branco/cinza/amarelo). Paletas 1 e 2 ficam livres.
- A CHR é **RAM**: `INITCHR $826F` espera o vblank, desliga a tela, zera e copia os 8 KB da fonte, e religa (`$82F0: LDA #$0A / STA $2001`).
- Texto: tudo é desenhado no buffer `$6000` e enviado por `GFX_FULL $82FA` (redesenho completo), `REFRESH $8326` (128 bytes por vblank, usado pelos menus) ou `UPDCUR $83E6` (só a coluna do cursor).
- O NMI do OS gasta ~315–330 ciclos; sobram **~1.850 ciclos** úteis de vblank (NTSC, 2.273 no total).

4. Ponteiros do OS base (build do clone, SHA-1 dbea89d3…)

4.1 Rotinas

Endereço	Nome	O que faz / como chamar
`$8000`	reset	init do hardware, `copydata`/`zerobss`, `c_sp=$0800`
`$80B4`	nmi	lê o joypad se `$A2==0`; põe `$9E`
`$826F`	`INITCHR` / ppu_screen_init_upload_chr	espera vblank, desliga a tela, zera e copia a CHR, religa
`$82FA`	`GFX_FULL`	envia a nametable inteira do buffer `$6000`
`$8326`	`REFRESH`	redesenho progressivo (menus/popups)
`$833D`	(fim do REFRESH)	`JMP $8343` — ponto do gancho `hide`
`$8393`	`WAITVB` / ppu_wait_vblank	espera o vblank lendo `$2002`
`$839C`	`PUTS`	
`$83C5`	`SCR_CLEAR`	zera `$6000–$63FF`
`$83E6`	`UPDCUR`	atualiza só a coluna do cursor
`$8441` / `$845F`	scr_backup / scr_restore	copia 1 KB da tela para `$6400` e volta
`$84C0`	flash_cmd_stub_save_ram	**chamado sempre no boot** (`$BBB1`): usa `$6400–$66FF` como rascunho
`$8DC2`	fat_dir_scan	lê a pasta para o cache; **para em 254 entradas** (`0xfd < count`)
`$9254`	dir_get_entry	nome/infos da entrada N
`$96B3`	`CL2LBA` / fat_cluster_to_lba	u32 cluster → LBA (A/X/sreg)
`$9DE0`	`FINDNAME` / file_find_by_name	(u32 dir, "NOME8 EXT", u32* tam, u32* cluster); remove 10 bytes da pilha do C
`$9F0B`	`WAITKEY`	
`$A5C8`	txt_print_xy	
`$AB25`	`IDLE`	espera ociosa do laço do navegador (ponto do gancho `idle_hook`)
`$BB4A`	(os_init)	`JSR FINDNAME` do `MAPROUT.BIN` → gancho `init_hook`
`$BBB1`	(os_init)	`JSR $84C0` sem condição
`$B255`	verificação anti-clone	não mexer
`$C53F`	`RDSEC` / fat_read_sector_cached	u32 LBA → `*($04E9)`; cache do LBA em `$0529–$052C`
`$C822`	ordenação da lista	
`$C987`	browser_draw_list	desenha a página (21 itens no original)
`$CB29`	(draw)	`LDA #$1D`: largura do nome na lista (29 → 14 colunas)
`$CB83`	(draw)	`JSR $82FA`: redesenho completo → gancho `w82fa` (texto)
`$CB8B`	(draw)	`LDA #$0A / STA $2001` → gancho `mask_entry`
`$CB94`	(draw)	`JSR $83E6` → gancho `hook_cursor`
`$D953`	laço do navegador	
`$D9CD`, `$D9FB`	repetição do direcional	atraso de 20 quadros (não é o tamanho da página)
`$DA06–$DBB0`	navegação	↑ ↓ ← → (página, linha, wrap)
`$ECDB`, `$ED7B`	(main)	`JSR $826F` → `chr_all` (fonte em todos os bancos)
`$ED50`	main	

Runtime do cc65 usado: `pushax $F3F7`, `push0 $F3F3`, `pusheax $F21B`, `ldaxysp $F145`, `staxysp $F4AB`, `udiv $F54F`, `umul $F36D`.

4.2 Variáveis e dados

Endereço	O que é
`$8259`/`$825A`	índice selecionado (16 bits, posição na lista ordenada)
`$0549`/`$054A`	número de itens da pasta

Endereço	O que é
$0515	joypad (botões)
$04E9	ponteiro para o buffer de setor do FAT (FATBUF)
$0529–$052C	LBA do setor no cache
$04FF–$0502	cluster da pasta /edfc
$6320–$635F	nome completo do item selecionado (linhas 25–26 do buffer) — base do CRC da capa
$FB39	tabela de ponteiros do menu (copiada para a RAM)
$F6CF	string "About"
$F753–$F895	strings do Device Info / About (reaproveitadas: SEG_C)
$D0C6–$D329	código das telas Device Info + About (reaproveitado: SEG_MAIN)

5. Arquitetura da firmware v15

5.1 Mapa completo dos patches (diferença byte a byte v15 × original)

Endereço	Original → v15	Função
$81C5–$820C	72 bytes (rotina de checksum **sem chamador**)	SEG_A: hook_cursor, draw_hook (filtro .nes), no_cover, mask_entry, idle_hook
$82EC, $841B, $BB9C	80 → A0	PPUCTRL com sprites 8×16
$8400	84 → A4	idem (modo de coluna)
$833D	JMP $8343 → JSR $F753	hide: esconde a capa quando um menu redesenha
$A680–$A706	135 bytes	SEG_E: split, frm_tg, i_wait, espq **em** $A706
$BB4A	JSR $9DE0 → JSR $D0C6	init_hook: acha MAPROUT.BIN **e** COVERS.BIN
$C9A4/$C9AC/$C9DC	15 → 14	página de 20 itens (desenho)
$CB2A	1D → 0E	nome com 14 colunas na lista
$CB84	JSR $82FA → JSR $F879	w82fa: escreve @tavinho.games antes do redesenho
$CB8B	LDA #$0A/STA $2001 → JSR $81EE/NOP/NOP	mask_entry
$CB95	JSR $83E6 → JSR $81C5	hook_cursor
$CCE2, $CCFD	Device Info / About → $D215	as duas opções do menu viram RTS (cm_r)
$D0C6–$D326	609 bytes	SEG_MAIN (ver 5.2)
$D9EC	JSR $AB25 → JSR $81F9	idle_hook
$DA10 $DA18 $DA49 $DA9F $DAE6 $DAFD $DB56 $DB63 $DB6B	-1	página de 20 (navegação)
$DB17, $DB73	LDY #3/JSR $F145 → LDA #0/TAX/NOP/NOP	← →: linha 0 na página nova
$DBA2	LDA $8259/SEC/SBC #$15 → LDY #4/LDA ($28),Y/SBC #$14	←: início da página − 20
$ECDC, $ED7C	JSR $826F → JSR $D216	chr_all: fonte em todos os bancos de CHR
$F753–$F894	322 bytes	SEG_C (ver 5.2)
$FBD4–$FBFE	43 bytes	SEG_D: scroll0, banco, dly, setP
arquivo 0x1000–0x15FF, 0x1EF0–0x1FFF	CHR tabela 1	**resto da capa de teste da PoC v1** (sem uso desde a v4; inofensivo — a limpeza foi tentada na v12b e abandonada)

5.2 Segmentos residentes e espaço livre

Segmento	Faixa	Usado / total	Livre	Conteúdo
SEG_A	$81C5–$820F	72 / 75	3	ganchos do desenho e da espera
SEG_E	$A680–$A706	135 / 135	0	split, troca de quadro do flicker, espera (espq)
SEG_MAIN	$D0C6–$D329	609 / 612	3	init_hook, crc_part, load_cover, lmod, chr_all, xvb/xvb_fim, ld_oam, nt_rows, q_next
SEG_C	$F753–$F895	322 / 323	1	hide, bg_clear, lba_read, variáveis, flk_vb, lm_mg, w82fa + texto
SEG_D	$FBD4–$FBFF	42 / 44	2	scroll0, banco, dly, setP
Módulo (WRAM)	$6400–$67FB	983 / 1020	37	carga da capa, sprites, atributos, paletas, flicker, cancelamento

O espaço residente está praticamente **esgotado**; código novo deve ir para o módulo (37 bytes livres) ou exigir liberar outra área (ver as notas técnicas, seção 3 e 11, para candidatas).

5.3 Endereços principais do código novo (v15)

Endereço	Símbolo	Papel
$81C5	hook_cursor	no lugar do JSR $83E6: cursor andou → marca a capa como pendente
$81C8	draw_hook	filtro: o nome termina em .nes? senão no_cover ($81DE)
$81EE	mask_entry	PPUMASK do redesenho (com ou sem sprites)
$81F9	idle_hook	a cada volta ociosa: i_wait → load_cover
$A680	split	troca dos bancos de CHR durante o desenho (sprite 0 na linha 16, depois de 24 em 24 linhas)
$A6ED	frm_tg	alterna o quadro do flicker
$A6F6	i_wait	conta os quadros com o cursor parado
$A706	espq	**quadros de espera + 1** (offset **34566** no OS.BIN; padrão 31 = 0,5 s)
$D0C6	init_hook	boot: acha MAPROUT.BIN e /EDFC/COVERS.BIN (cluster → LBA base)
$D12E	crc_part	CRC-32 do nome (linhas 25–26)
$D19A	load_cover	busca no índice e carrega a capa
$D1B8 / $D1BA	lmod / lm_k	confere o módulo na WRAM (LDA $66FC,X na v15) e relê os setores 257–258 se precisar
$D216	chr_all	$826F nos bancos 0, 1, 3 … 13 (a fonte em todos os bancos usados)
$D240 / $D28E	xvb / xvb_fim	abre/fecha a janela de vblank estendido
$D2AE	ld_oam	monta a OAM da carga
$D2E2	nt_rows	nametable da capa (escondida)
$F753	hide	esconde a capa (menus), regrava as paletas do OS
$F799	bg_clear	limpa as células da capa no buffer
$F7B7	lba_read	lê um setor do COVERS.BIN (base + deslocamento)
$F7E5…$F808	variáveis	base, want, cur, sec, csec, pending, has, bgshown, rev, …
$F83E	flk_vb	no vblank: paletas e OAMDMA do quadro do flicker
$F875	lm_mg	assinatura do módulo (4 bytes = módulo[$2FC:$300] = 04 8D 55 7F)
$F879	w82fa	copia "@tavinho.games" para $62F1 (linha 23, colunas 17–30) e salta para $82FA
$FBD4	scroll0	zera o scroll
$FBE2	banco	escreve $82 no registrador de CHR
$FBEB	dly	atraso calibrado (janela)
$FBF3	setP	ponteiro
$6401	lc_rest	começo da carga no módulo (guarda a pilha em ab_sp $7F5D)
$65A5	up128	envia 128 bytes à PPU (laço desenrolado, ~11,5 ciclos/byte)
$65FE	at_ppu	atributos da capa direto na PPU (cópia em ATC $7F00)
$6629	pal_wr	paletas (inclui a cor 3 da paleta 3, $3F0F)
$670F	spr_load	carrega os tiles dos sprites (faixas A e B)
$6773	vbw	espera o vblank pelo flag $9E do NMI do OS
$67A7	joyp	lê $4016 direto (o NMI fica desligado nas janelas)
$67BD	l_desiste	cancela: restaura a pilha, religa a tela, pending=1

A tabela completa (242 símbolos) está no **Apêndice A**.

5.4 Fluxo de execução

1. **Boot** — os_init chama init_hook ($BB4A): faz a busca original do MAPROUT.BIN e procura COVERS BIN na pasta /edfc, guardando o LBA do primeiro setor. Sem o arquivo, a firmware se comporta como o OS original (sem capas). main chama chr_all no lugar do $826F: a fonte é copiada para todos os bancos de CHR usados pela capa.
2. **Desenho da lista** — browser_draw_list termina em hook_cursor/draw_hook: se o item selecionado é .nes, a capa fica **pendente** (pending = 1), senão é escondida.
3. **Espera** — a cada volta ociosa do navegador, idle_hook → i_wait: pending vira contador de quadros e a carga começa quando chega em espq (o OS volta pending para 1 a cada movimento do cursor).
4. **Carga** — load_cover: crc_part calcula o CRC do nome (linhas 25–26 do buffer, |$20), lê o setor do balde (crc & $FF) do índice, acha o número da capa e confere os 3 bytes de verificação do registro. lmod garante o módulo na WRAM (confere $66FC–$66FF contra lm_mg; se não bater, relê os setores 257–258). O módulo carrega: setor 0

(paletas, atributos, nametable), grupos de tiles do fundo nos bancos ímpares, tiles de sprite nos bancos pares, OAM e paletas dos sprites — tudo dentro das janelas de vblank estendido.

5. **Exibição** — a cada quadro, `split` troca o banco de CHR em 6 pontos da tela (sincronizado pelo sprite 0 na linha 16); com flicker, `flk_vb`/`frm_tg` alternam as paletas e os tiles dos sprites a cada quadro.
6. **Saída** — qualquer menu/popup redesenha via `REFRESH $8326` e passa por `hide` (`$833D`): sprites desligados, paletas do OS regravadas. Botão durante a carga → `joyp` → `l_desiste`.

5.5 Técnicas de vídeo (por que funciona)

- **Mais tiles que a PPU permite:** a capa precisa de ~400 tiles; cada banco de 4 KB tem só 63 slots livres na fonte (`$01–$1F` e `$80–$9F`). Solução: 7 grupos de linhas, cada um num banco (`$82` = 1, 3, 5 … 13), com a fonte copiada em todos; o `split` troca de banco durante o desenho. As linhas de fronteira existem nos dois bancos vizinhos (a troca pode cair em qualquer ponto delas).
- **Sincronia pelo sprite 0:** um sprite opaco atrás do texto (coluna 0) marca a linha; o código espera o flag V de `$2002`.
- **Vblank estendido na carga (`xvb`):** espera o sprite 0 da carga na linha 200, atrasa até a linha 226 (abaixo da barra, que acaba na 223), desliga a tela **no intervalo horizontal** (ponto ~280; no meio da linha a PPU pode corromper a OAM) e o NMI (`$2000=$20`), envia vários pedaços de 128 bytes, e `xvb_fim` religa antes do pré-render. O NMI fica desligado porque ele lê `$2002` e zeraria o latch do `$2006` no meio das escritas.
- **Orçamento de ciclos:** toda escrita na PPU com a tela ligada precisa acabar dentro do vblank (~1.850 ciclos depois do NMI do OS). Os testes medem isso instrução por instrução.
- **Flicker:** quadro A e quadro B dos sprites (tiles `2p+1 EOR $80`, paletas próprias) alternando a 60 Hz; o olho vê a média — dobra as combinações de cor.
- **Atributos da capa fora do buffer do OS** (`ATC $7F00`, enviados direto à PPU): o buffer de atributos é só do OS, senão a limpeza da capa apagava o cinza das caixas de menu (bug da v10).

6. Formato do `COVERS.BIN`

Setores de 512 bytes. O arquivo tem que estar **contínuo no cartão**: a firmware lê "LBA base + deslocamento", sem seguir a cadeia de clusters do FAT.

Setores	Conteúdo
0–255	**índice**: um setor por balde (`crc & $FF`); entradas de 5 bytes `[crc>>8, crc>>16, crc>>24, idx_lo, idx_hi]`, fim = `idx_hi $FF`; até 101 entradas por balde (`PER_BUCKET`)
256	zerado
257–258	**módulo** de 1 KB (vai para `$6400–$67FF`)
512 + 64·i …	**registro da capa i** (64 setores = 32 KB)

- **CRC:** CRC-32 do zlib **sem o XOR final**, sobre o nome como o navegador mostra nas linhas 25–26 (≤ 60 caracteres), cada byte `| $20` (`gerar_covers.name_crc`).
- **Limite de 1015 capas:** o setor é de 16 bits no 6502 (`512 + 64·1014 + 63 < 65536`).
- **Registro (v8–v15):** setor 0 = verificação (3 bytes) + paletas 1 e 2 (`[3:9]`) + atributos + nametable + pedaços usados por grupo (`[353:367]`) + cor 3 da paleta 3 (`[367]`); setores 1–14 = tiles do fundo (7 grupos); setor 15 = OAM A (`[0:256]`) + paletas A (`[256:272]`) + faixas A (`[272:300]`) + flag do flicker (`[300]`) + paletas B (`[304:320]`) + faixas B (`[320:]`); setores 16+ = tiles dos sprites em faixas contínuas. A referência exata é `conv_capa_grande.converter_v9` / `converter_v8`.
- **Assinatura do módulo:** v15 compara `módulo[$2FC:$300]` (`048d557f` no `MOD_v10.BIN`). O mesmo `COVERS.BIN` serve para v10, v13, v14 e v15 (o módulo é o mesmo).
- **Modo "só acrescentar"** (`gerar_covers_v10.py --base`): lê o índice antigo, copia os registros byte a byte, converte só as ROMs novas, acrescenta no fim e refaz o índice; recusa se o módulo for de outra firmware.

7. Os patches das últimas versões em detalhe

7.1 Texto `@tavinho.games` (v13)

`$CB83: JSR $82FA` → `JSR w82fa` (`$F879`): `LDX #13 / LDA s_txt,X / STA $62F1,X / DEX / BPL / JMP $82FA` — copia 14 caracteres para a linha 23, colunas 17–30 do buffer, antes de o OS enviar a tela. Os atributos da capa já põem as linhas 22–23 na paleta 0 (texto), então ele aparece com e sem capa.

7.2 20 jogos por página (v13) — 13 operandos trocados no lugar

Endereço (opcode)	Antes → depois	Papel
`$C9A3 LDA #` / `$C9AB LDA #`	21 → 20	início da página = sel/21·21
`$C9DB CMP #`	21 → 20	laço das linhas
`$DA0F` / `$DA17 LDA #`	21 → 20	início da página (navegação)
`$DA48 LDA #`	20 → 19	↑ na 1ª linha vai para a última
`$DA9E CMP #`	21 → 20	↓ depois da última
`$DAE5 CMP #`	22 → 21	← → só com mais de uma página
`$DAFC LDA #`	21 → 20	→ avança uma página
`$DB55 CMP #` / `$DB62` / `$DB6A LDA #`	21 → 20	← da 1ª página vai para a última
`$DBA6 SBC #`	21 → 20	← volta uma página

Não confundir com `$D9CD/$D9FB` (atraso da repetição, 20 quadros) nem com `$858D/$85A1`, `$920B`, `$B954`, `$CCBA`, `$CF76` (outros 21/20 que não são da página).

7.3 ← → no topo da página (v14)

O OS faz `sel = início da página nova + linha` (a linha fica em `c_sp[2..3]`). `$DB17` e `$DB73`: `A0 03 20 45 F1` (`LDY #3 / JSR ldaxysp`) → `A9 00 AA EA EA` (`LDA #0 / TAX / NOP / NOP`). `$DBA2`: `AD 59 82 38` (`LDA $8259 / SEC`) → `A0 04 B1 28` (`LDY #4 / LDA ($28),Y`, o início da página; o carry já é 1).

7.4 Correção do travamento após ligar (v15)

O módulo mora na WRAM **com bateria** e sobrevive ao desligar. Em todo boot, `os_init` (`$BBB1`) chama `$84C0`, que usa `$6400–$66FF` como rascunho e estraga os primeiros 768 bytes do módulo — mas não toca em `$67FC`, onde ficava a assinatura. Assinatura intacta + corpo estragado → o `lmod` não relia e a primeira carga saltava para lixo (trava). Abrir um jogo apagava a WRAM inteira, por isso "funcionava num boot e travava no outro". Correção (3 bytes + a assinatura, sem mudar o módulo): `lm_k` (`$D1BA`) `BD FC 67` → `BD FC 66` (`LDA $66FC,X`) e `lm_mg` = `módulo[$2FC:$300]`.

8. Conversor de imagens (PC)

Etapa	Arquivo / função	Algoritmo
Associação ROM ↔ capa	`associar_roms.py`	títulos "limpos" (sem número de lista, `(...)`/`[...]`, "The", acentos; II..VIII = 2..8), `difflib` com limiar 0,84; números do título têm que bater; empate decide pela região da ROM (senão EUA > Mundo > Europa > Japão) e evita Beta/Proto/Pirata. Várias ROMs podem usar a mesma imagem
Recorte e redução	`gerar_covers_v10` / `sim_tela_cheia`	corte central na proporção 0,7, redução para 112×160
Nitidez	`sim_tela_cheia` (`UnsharpMask`, raio 1) + `conv_v3` (configuração por capa)	força por capa de 10–200% (padrão 180 nas POCs)
Fundo	`conv_capa_grande.quantizar`	2 paletas livres (3 cores + preto) escolhidas para a capa, uma paleta por bloco 16×16; opcional a cor 3 da paleta 3 (`converter_v9` fica com o melhor dos dois)
Pontilhado	`conv_capa_grande.pontilhar`	xadrez Bayer 2×2: cada ponto mistura 2 cores do bloco em 0/25/50/75/100%; custo $\Delta E^2 + 0{,}05\cdot\Delta E^2(c1,c2)\cdot 4t(1-t)$
Sprites	`conv_sprites_grande.py`	até 62 sprites 8×16 (grade de 16 linhas × 8 pontos, máx. 8 por faixa, 7 na faixa do sprite 0), 4 paletas; cada ponto transparente ou cor, só onde reduz o erro
Flicker	`conv_sprites_grande.otimizar_flicker`	quadro B por sprite; ponto = média na luz de A e B; custo $\Delta E^2 + \lambda\cdot\Delta E^2(A,B)$; 7 níveis λ = 0 / 0,02 / **0,05** / 0,10 / 0,15 / 0,20 / 0,25. A busca testa as 54 candidatas de uma vez (`busca()`): ~8× mais rápido, resultado idêntico
Edição de cores	`capa_grande_cores.py`	recolorir = só trocar bytes de paleta (setor 0 `[3:9]`, `[367]`; setor 15 `[256:272]`, `[304:320]`); prévia idêntica ao conversor
Erro de cor	`sim_tela_cheia.lab`, `palette_lab/nes_core.rgb2lab`	ΔE no espaço CIELAB, sobre as cores RGB da paleta do NES (`build_poc.NES_RGB`)

Tempo típico: ~1 s por capa sem flicker, ~12 s com flicker (7 níveis calculados em segundo plano na prévia).

9. A ferramenta (`ferramentas/gerador_firmware_capas.py`)

- **Interface:** tkinter. Seções: (1) capas, ROMs, saída e "Só acrescentar"; (2) prévia da capa (espera, nitidez, pontilhado, flicker + nível, paletas/conta-gotas, zoom, barra de progresso); (3) gerar e copiar para o SD.
- **Firmware de referência:** `firmwares/OS_v15.BIN` + `firmwares/MOD_v10.BIN`, conferidos pelo SHA-1. A espera é o **único byte** que a ferramenta muda no OS (`ESPQ = 34566`, valor = quadros + 1).
- **Configurações:** `gerador_config.json` (ao lado da ferramenta, ou em `%APPDATA%\NES N8 Covers editor\` no .exe); por pasta de capas, `capas_config.json` (nitidez, flicker, nível, pontilhado por capa) e `paletas_editadas_grande/` (edições de cor). Cache das conversões com flicker em `cache_flicker/`.
- **Geração:** `gerar_covers_v10.py` (também roda sozinho pela linha de comando). No .exe roda num `multiprocessing.Process` do próprio programa (`linhas_gerador`/`_gerador_processo`), com `freeze_support()`; fora dele, como subprocesso Python (`GERADOR_EM_PROCESSO=1` força o modo do .exe).
- **Cópia para o SD:** `copiar_sd.py` apaga o antigo, reserva o tamanho inteiro, grava, confere com `fsutil file queryextents` (1 trecho = contínuo) e só então troca.
- **.exe:** `compilar_exe.py` (PyInstaller `--onedir --windowed`, ícone `icone.ico`, embute `OS_v15.BIN`, `MOD_v10.BIN` e o ícone; preserva o `LEIA-ME.txt`; gera o .zip). Modos escondidos para teste: `--gerar-covers …` e `--testar-geracao <log> …`.

10. Testes (no PC, antes do aparelho)

Infraestrutura própria:

- `emu6502.py` — emulador 6502 (opcodes oficiais) com *stubs* para rotinas do OS e *hooks* de leitura/escrita.
- `ciclos6502.py` — ciclos por instrução (inclui página cruzada).
- `ppu_tempo.py` — **modelo de tempo da PPU** (NTSC 262×341): NMI de verdade do OS na linha 241, sprite 0, `$2002` apagando vblank e latch, `WAITVB` como salto até o vblank, joypad lido pelo NMI, botão a partir de um instante (`joy_em`). Qualquer escrita na PPU com a tela ligada fora do vblank é uma **violação**.
- `asm6502.py` (montador com símbolos e expressões) e `dis6502.py` (disassembler).

Teste	Verifica
`teste_capas_v10_emu.py`	tempo real da v10–v15 (`main('build_poc_capa_grande_v15', …)`): carga, tela = conversor, nenhuma escrita fora do vblank, espera, cancelamento, assinatura do módulo
`teste_ligar_v15.py`	WRAM com o módulo da sessão anterior e `$6400–$66FF` estragados: v14 trava, v15 relê e carrega
`teste_pagina_v14.py` / `teste_pagina_v12.py`	roda a navegação `$DA06–$DBB0` do próprio OS para listas de 1 a 254 itens, toda posição, 4 direções (12.232 casos)
`teste_abrir_pasta.py`	abrir pastas no emulador
`teste_gerador_gui_v10.py`	a janela de verdade: prévias, opções salvas, geração = POC idêntica (`COVERS.BIN 0e495260…`), espera muda só o byte `espq`
`teste_capas_emu.py`, `teste_capas_v2..v9_emu.py`, `teste_poc_*.py`, `teste_paletas_editadas.py`	histórico das versões anteriores

Os testes de capa usam 5 capas reais em `imagens/Named_Boxarts/` (libretro-thumbnails; não incluídas) e o OS original em `firmware_base/EDFC/OS.BIN` (não incluído).

11. Bugs encontrados, causas e lições

Sintoma no aparelho	Causa real	Lição
Lixo ao trocar de capa (v5)	escritas na PPU passavam do vblank (3 pares por vblank + laço das máscaras: até 2.203 ciclos)	medir ciclos de **toda** escrita na PPU; 2 pares por vblank
Lixo na linha 0 / na lista (1ª v7)	a janela atravessava a linha 241; o NMI do OS lia `$2002` e zerava o latch do `$2006`	NMI desligado na janela; sincronizar pelo sprite 0; testar com o NMI **de verdade** (`ppu_tempo`)
Travava ao parar num jogo (2ª v7)	módulo velho na WRAM com a mesma marca "M7OK"	assinatura derivada do conteúdo do módulo
Caixa do File Menu pela metade (1ª v10)	a limpeza da capa zerava atributos no buffer do OS	estado da capa fora das estruturas do OS (`ATC`)
Travava ao abrir pasta (v12)	atribuído à fragmentação do `COVERS.BIN` e depois a T+M+P; a causa mais provável era a do v15	isolar com builds C1/C2 ajudou, mas a hipótese "fragmentação" foi aceita cedo demais
Lixo de ~0,5 s ao ligar	`chr_all` troca de banco com a tela ligada entre chamadas do `$826F`	correção (v12b) quebrou as capas; **cosmético, deixado como está**

Sintoma no aparelho	Causa real	Lição
Travava depois de ligar, alternando entre boots (v10–v14)	o OS estraga `$6400–$66FF` (WRAM com bateria) em todo boot; a assinatura ficava fora dessa faixa	conferir a assinatura **dentro** da área que pode ser estragada; testar "cold boot" com WRAM suja
Nome "colado" (`contanes`) e sem capa	ROM com nome todo em minúsculas (o navegador mostra o nome errado, o CRC não bate)	exigir nome com maiúscula no início

Lições gerais: emular o **tempo** (não só a lógica) da PPU; toda memória persistente (WRAM com bateria) pode estar suja; mudanças pequenas e isoláveis por versão; backup estável antes de cada experimento.

12. Limites e quirks

Limite	Valor	Origem
Capas por `COVERS.BIN`	1015	setor de 16 bits na firmware
Itens por pasta	254	`fat_dir_scan` do OS original
Tipos com capa	só `.nes`	filtro `draw_hook`
Nome da ROM	≤ 60 caracteres, sem acentos, começa com maiúscula	linhas 25–26 (2×30) + CRC; nome todo minúsculo aparece errado
Tamanho	32 KB por ROM (817 ROMs ≈ 26 MB)	registro de 64 setores
Espera	0–4 s	byte `espq`
Paletas fixas	paleta 0 (lista), branco/cinza da barra, preto	compartilhadas com o OS

Quirks herdados do OS original: com 256+ itens ← → não trocam de página (o `BCS` de `$DAE7` usa o carry do `CMP #4` de `$DA8D`) e a última página sai cheia; ← na 1ª página com n múltiplo de 20 cai numa página vazia e o cursor vai para o último jogo.

13. Aplicações e ferramentas usadas

Ferramenta	Uso
Ghidra 12.1.4 (headless, processador `6502:LE:16:default`, base `$8000`)	disassembly/decompilação do OS; `ghidra_scripts/DumpNES.java` + `nomes_base.txt` renomeiam ~180 funções e exportam listagem, pseudo-C e referências de I/O
REA (MCP de engenharia reversa)	tentado primeiro; **não suporta 6502** (formato/arquitetura recusados)
Python 3.15 + **numpy** + **Pillow** + **tkinter**	ferramenta, conversor, builders, testes
Emulador/montador próprios	`emu6502.py`, `asm6502.py`, `dis6502.py`, `ciclos6502.py`, `ppu_tempo.py`
Conhecimento do **cc65**	convenções de chamada (pilha de software em `$28`, `pushax`, `ldaxysp`), runtime em `$ED50–$F532`
`map_funcs.py`	casa as funções do v1.26 (krikzz) com o build do clone (169 funções)
PyInstaller 6.22.3	o .exe (`compilar_exe.py`)
ffmpeg	extrair quadros dos vídeos do aparelho (diagnóstico do lixo ao ligar)
`fsutil file queryextents` (Windows)	conferir se o `COVERS.BIN` ficou contínuo
reportlab	este PDF
Palette Lab (próprio)	laboratório de paletas da capa pequena (v1–v3)
libretro-thumbnails *Named_Boxarts*	fonte das capas de teste
Claude Code	análise, código e testes junto com o autor
Hardware	EverDrive N8 clone + NES NTSC; fotos/vídeos em `imagens/` e `debug/`

14. Como reconstruir tudo

1. Python 3 com `numpy` e `Pillow` (`pip install -r requirements.txt`).
2. Rodar a ferramenta: `python ferramentas/gerador_firmware_capas.py` (usa `ferramentas/firmwares/OS_v15.BIN`).
3. Reconstruir a firmware a partir do zero: copie o **seu** `OS.BIN` original do clone (SHA-1 `dbea89d3…`) para `firmware_base/EDFC/OS.BIN` e rode `python ferramentas/build_capas_v15.py` (mostra o SHA-1; tem que dar `caa12ed7…` com a espera padrão). `build_poc_capa_grande_v15.py` monta uma pasta `EDFC` de teste (precisa das capas em `imagens/Named_Boxarts`).

4. Testes: `python ferramentas/teste_ligar_v15.py`, `python ferramentas/teste_pagina_v14.py`, e o tempo real `python -c "import teste_capas_v10_emu as T; T.main('build_poc_capa_grande_v15', 'v15.png')"` dentro de `ferramentas/`.
5. .exe: `pip install pyinstaller` e `python ferramentas/compilar_exe.py`.

O histórico completo da investigação (seções 0–39, com todas as versões) está em `NOTAS-TECNICAS-FIRMWARE.md` (**Apêndice B** deste PDF).

Apêndice A — Tabela de símbolos da firmware v15

Gerada pelo montador (`asm6502`) ao construir a v15 (`build_capas_v15.main()`): 242 símbolos. Rótulos de código/variáveis por endereço e, depois, as constantes/equates (rotinas do OS, registradores, endereços fixos).

End.	Símbolo	Região
$6400	l_fail1	módulo (WRAM)
$6401	lc_rest	módulo (WRAM)
$6412	l_scan	módulo (WRAM)
$642E	l_found	módulo (WRAM)
$6438	l_sh	módulo (WRAM)
$6462	l_pc	módulo (WRAM)
$646F	l_at	módulo (WRAM)
$648E	l_n0	módulo (WRAM)
$6498	l_nr	módulo (WRAM)
$649A	l_nc	módulo (WRAM)
$64AC	l_n1	módulo (WRAM)
$64B4	l_ct	módulo (WRAM)
$64C9	l_g	módulo (WRAM)
$651D	l_spb	módulo (WRAM)
$6548	l_sec	módulo (WRAM)
$6550	l_s0	módulo (WRAM)
$6555	l_s1	módulo (WRAM)
$656F	l_fr	módulo (WRAM)
$6577	l_fok	módulo (WRAM)
$659B	l_fe	módulo (WRAM)
$65A1	l_sr	módulo (WRAM)
$65A2	l_serr	módulo (WRAM)
$65A5	up128	módulo (WRAM)
$65B3	l_up	módulo (WRAM)
$65B4	src	módulo (WRAM)
$65BB	src2	módulo (WRAM)
$65D1	l_c1	módulo (WRAM)
$65DF	l_c2	módulo (WRAM)
$65E0	sp_hide	módulo (WRAM)
$65FE	at_ppu	módulo (WRAM)
$6600	ap_r	módulo (WRAM)
$660E	ap_c	módulo (WRAM)
$6618	ap_w	módulo (WRAM)
$6629	pal_wr	módulo (WRAM)
$6659	pw_c	módulo (WRAM)
$665D	pw3	módulo (WRAM)
$665F	pw_l	módulo (WRAM)
$6669	pw_w	módulo (WRAM)
$6671	reveal	módulo (WRAM)
$6671	rv_fim	módulo (WRAM)
$6688	rv_sp	módulo (WRAM)
$669E	sp_all	módulo (WRAM)
$66A8	sp_g	módulo (WRAM)
$66CD	sp_d	módulo (WRAM)
$66DC	sp_s	módulo (WRAM)
$66E5	sp_l	módulo (WRAM)
$66F7	sp_n	módulo (WRAM)
$670E	sl_r0	módulo (WRAM)
$670F	spr_load	módulo (WRAM)
$6732	sl_m	módulo (WRAM)
$6746	sl_d	módulo (WRAM)
$6752	sl_f	módulo (WRAM)
$6752	sp_oam	módulo (WRAM)
$676A	sp_o	módulo (WRAM)
$6772	sp_r	módulo (WRAM)
$6773	vbw	módulo (WRAM)

End.	Símbolo	Região
$677E	vb1	módulo (WRAM)
$678D	vb_r	módulo (WRAM)
$678E	flk_tg	módulo (WRAM)
$6795	ft_l	módulo (WRAM)
$67A6	ft_r	módulo (WRAM)
$67A7	joyp	módulo (WRAM)
$67B2	jp_l	módulo (WRAM)
$67BC	jp_r	módulo (WRAM)
$67BD	l_desiste	módulo (WRAM)
$7F50	wcnt	variáveis WRAM
$7F51	pair	variáveis WRAM
$7F52	scnt	variáveis WRAM
$7F53	smk	variáveis WRAM
$7F54	sbank	variáveis WRAM
$7F55	gi	variáveis WRAM
$7F56	nch	variáveis WRAM
$7F57	ntot	variáveis WRAM
$7F58	vbt	variáveis WRAM
$7F59	xsx	variáveis WRAM
$7F5A	xfb	variáveis WRAM
$7F5C	frm	variáveis WRAM
$7F5D	ab_sp	variáveis WRAM
$81C5	hook_cursor	SEG_A
$81C8	draw_hook	SEG_A
$81CC	f_loop	SEG_A
$81DB	f_skip	SEG_A
$81DE	no_cover	SEG_A
$81E6	f_yes	SEG_A
$81EE	mask_entry	SEG_A
$81F9	idle_hook	SEG_A
$8206	i_done	SEG_A
$8209	ext	SEG_A
$A680	split	SEG_E
$A685	sp_out	SEG_E
$A688	sw_hd	SEG_E
$A68B	sp_go	SEG_E
$A68E	sw_v	SEG_E
$A6A3	sw_1	SEG_E
$A6A5	sw1a	SEG_E
$A6B2	sw2	SEG_E
$A6B4	sw2a	SEG_E
$A6C1	sw3	SEG_E
$A6CD	sw_l	SEG_E
$A6ED	frm_tg	SEG_E
$A6F6	i_wait	SEG_E
$A6FF	iw_1	SEG_E
$A706	espq	SEG_E
$D0C6	init_hook	SEG_MAIN
$D0DA	in_clr	SEG_MAIN
$D12C	in_done	SEG_MAIN
$D12E	crc_part	SEG_MAIN
$D132	c_ini	SEG_MAIN
$D139	c_loop	SEG_MAIN
$D146	c_bit	SEG_MAIN
$D168	c_nb	SEG_MAIN
$D16B	c_next	SEG_MAIN

End.	Símbolo	Região
$D172	c_cmp	SEG_MAIN
$D187	c_diff	SEG_MAIN
$D189	c_st	SEG_MAIN
$D196	c_hide	SEG_MAIN
$D199	l_fail0	SEG_MAIN
$D19A	load_cover	SEG_MAIN
$D19C	l_cp	SEG_MAIN
$D1B8	lmod	SEG_MAIN
$D1BA	lm_k	SEG_MAIN
$D1C8	lm_go	SEG_MAIN
$D1D7	lm_s	SEG_MAIN
$D1E9	lm_c	SEG_MAIN
$D1EB	lm_d	SEG_MAIN
$D205	lm_r	SEG_MAIN
$D206	cmp3	SEG_MAIN
$D208	cm	SEG_MAIN
$D215	cm_r	SEG_MAIN
$D216	chr_all	SEG_MAIN
$D21B	ca_l	SEG_MAIN
$D22E	ca_2	SEG_MAIN
$D240	xvb	SEG_MAIN
$D24B	xw1	SEG_MAIN
$D258	xw2	SEG_MAIN
$D25A	xw3	SEG_MAIN
$D267	xw4	SEG_MAIN
$D26E	x_off	SEG_MAIN
$D286	x_fb	SEG_MAIN
$D28E	xvb_fim	SEG_MAIN
$D296	xf_0	SEG_MAIN
$D29B	xf_1	SEG_MAIN
$D2AE	ld_oam	SEG_MAIN
$D2B2	lo_1	SEG_MAIN
$D2E2	nt_rows	SEG_MAIN
$D2EF	nr_r	SEG_MAIN
$D2FE	nr_c	SEG_MAIN
$D31B	q_next	SEG_MAIN
$D326	qn_r	SEG_MAIN
$F753	hide	SEG_C
$F761	hd_pal	SEG_C
$F769	hd_p3	SEG_C
$F780	hide_draw	SEG_C
$F788	h_inv	SEG_C
$F793	h_mask	SEG_C
$F799	bg_clear	SEG_C
$F7A3	bc_r	SEG_C
$F7A5	bc_c	SEG_C
$F7AD	bc_k	SEG_C
$F7B7	lba_read	SEG_C
$F7D9	s_cov	SEG_C
$F7E5	base	SEG_C
$F7E9	want	SEG_C
$F7ED	cur	SEG_C
$F7F1	sec	SEG_C
$F7F3	csec	SEG_C
$F7F5	pdst	SEG_C
$F7F7	rq	SEG_C

End.	Símbolo	Região
$F7F9	cpal	SEG_C
$F7FF	enabled	SEG_C
$F800	pending	SEG_C
$F801	has	SEG_C
$F802	bgshown	SEG_C
$F803	rev	SEG_C
$F804	fs_t	SEG_C
$F805	grp	SEG_C
$F806	amode	SEG_C
$F807	cnt	SEG_C
$F808	nt_hid	SEG_C
$F83E	flk_vb	SEG_C
$F85B	fv_1	SEG_C
$F85D	fv_l	SEG_C
$F874	fv_r	SEG_C
$F875	lm_mg	SEG_C
$F879	w82fa	SEG_C
$F87B	wt_l	SEG_C
$F887	s_txt	SEG_C
$FBD4	scroll0	SEG_D
$FBE2	banco	SEG_D
$FBEB	dly	SEG_D
$FBEC	dl_i	SEG_D
$FBF3	setP	SEG_D

Constantes e equates

Nome	Valor	Nome	Valor	Nome	Valor
AT0	$63C4	ATC	$7F00	ATTR_BUF	$63CD

Nome	Valor	Nome	Valor	Nome	Valor
CAB	$7F80	CHT	$7F60	CL2LBA	$96B3
COR3	$7F6E	EDFC_CL	$4FF	FATBUF	$4E9
FINDNAME	$9DE0	FLAG	$7FAC	GFX_FULL	$82FA
IDLE	$AB25	INITCHR	$826F	JOY	$515
LDEAXI	$F14C	MOD	$7F00	MOD2	$6400
NAMEROWS	$6320	NMIF	$9E	NT0	$6050
NT_BUF	$6096	P	$34	PALTAB	$7F00
PUSH0	$F3F3	PUSHAX	$F3F7	PUSHEAX	$F21B
PUTS	$839C	Q	$36	RDSEC	$C53F
REFRESH	$8326	SCR_BACKUP	$8441	SCR_CLEAR	$83C5
SCR_RESTORE	$845F	SMASK	$7F10	SPAL	$7F00
SPAL8	$7F80	SPALB8	$7FB0	SREG	$2A
STB	$7F70	STB8	$7F90	STBB8	$7FC0
T0	$30	T1	$31	T2	$32
T3	$33	TP	$92	UPDCUR	$83E6
WAITKEY	$9F0B	WAITVB	$8393		

Apêndice B — Notas técnicas completas (NOTAS-TECNICAS-FIRMWARE.md)

O diário técnico do projeto, na ordem em que as coisas foram descobertas (seções 0–39). Endereços das seções 1–9 são do OS v1.26 do krikzz; a partir da seção 10 valem os do build do clone (base).

Análise estática do `OS.BIN` (57.344 bytes, 2021-04-04) com o objetivo de criar uma custom firmware com **capas de jogos** no navegador de arquivos, no mesmo espírito de `capas-everdrive-master` (SMS) e `Tavinho-edmd-covers` (Mega Drive).

> [OK] VERSÃO ESTÁVEL v2 — 2026-10-23 Firmware v2 (sprites + fundo) **testada no aparelho**: `OS.BIN` SHA-1 `c0a74aeab86b2ad3594b062535e4ef88cb866818` + `COVERS.BIN` v2 (gerado com `gerar_covers.py --v2` ou pelo Palette Lab). Inclui as ferramentas (Palette Lab com modo capa e conta-gotas). Backup completo em `ESTAVEL_v2/` e `ESTAVEL_v2.zip`. A favorita v1 continua abaixo.

> * MELHOR VERSÃO (favorito) — v1, 2026-10-23 Firmware com capas do SD **funcionando no aparelho**: `capas_sd/EDFC/OS.BIN` (SHA-1 `7c9f4886833a60dd9fddb69d44acd7637897fbd0`) + `COVERS.BIN` gerado por `ferramentas/gerar_covers.py`. Backup completo em `FAVORITO_melhor_versao_v1/` e `FAVORITO_melhor_versao_v1.zip`. Qualquer experimento posterior (ex.: 4 paletas extras no fundo, seção 14) parte desta versão e pode ser descartado voltando para ela.

Legenda de confiança: **[OBS]** observado diretamente no código/bytes · **[INF]** inferido com boa evidência · **[?]** hipótese, precisa de teste no hardware.

0. Ferramentas e como a análise foi feita

- O REA foi tentado primeiro. Ele **não abre código 6502**: recusa binário cru ("unsupported binary format") e, embrulhado num ELF com `e_machine=6502`, recusa com "unsupported ELF architecture" (a lista de arquiteturas é fixa no REA). A alteração temporária no Ghidra (um `6502.opinion`) foi **revertida**.
- Então usei o **Ghidra 12.1.4 em modo headless** (o mesmo motor que o REA usa), com o processador `6502:LE:16:default`, base `$8000`.
- Arquivos gerados:
  - `analise/ghidra_proj/EDFC_OS.gpr` — projeto Ghidra (abra no Ghidra GUI), com ~180 funções e variáveis renomeadas.
  - `analise/ghidra_out/listing.asm` — disassembly completo, anotado com nomes.
  - `analise/ghidra_out/decompiled.c` — pseudo-C de todas as 248 funções.
  - `analise/ghidra_out/functions.txt` — lista de funções + quem chama.
  - `analise/ghidra_out/xrefs_io.txt` — todo acesso a PPU/APU/registradores do cartucho.
  - `analise/strings_prg.txt` — strings com endereço de CPU.
  - `analise/chr_tabela0.png`, `analise/chr_tabela1.png` — as duas tabelas de padrões.
  - `analise/prg_8000.bin` (32 KB de PRG) e `analise/chr_0000.bin` (8 KB de CHR).
- Scripts (`ferramentas/`):
  - `dis6502.py OS.BIN <addr> <len> [run_addr]` — disassembler 6502 rápido (útil para stubs que são copiados para a RAM).
  - `chr2png.py OS.BIN out.png [offset] [tamanho]` — renderiza CHR sem dependências.
  - `ghidra_scripts/DumpNES.java` + `nomes_n8.txt` — refaz o projeto/exportações. Para adicionar nomes: edite `nomes_n8.txt` e rode de novo (comando no fim).

1. Layout do arquivo OS.BIN

Offset no arquivo	Tamanho	Conteúdo	Para onde vai
`0x0000–0x0FFF`	4 KB	CHR tabela 0: **duas cópias da fonte** (tiles `$00–$7F` cor normal, `$80–$FF` cor de destaque) [OBS]	PPU `$0000`
`0x1000–0x1FFF`	4 KB	CHR tabela 1: tiles `$00–$5F` sólidos, `$60–$AB` lixo, `$FF` logo pequeno. Contém o texto "ihor loves kitty (:" em `0x1AAD`. **Não é usada pelo menu** [OBS]	PPU `$1000`
`0x2000–0x3FFF`	8 KB	Tudo `$AA` (preenchimento) [OBS]	CPU `$4000–$5FFF`? [?]
`0x4000–0x5FFF`	8 KB	Tudo `$00` [OBS]	WRAM `$6000–$7FFF` [INF]
`0x6000–0xDFFF`	32 KB	**PRG** do OS (código 6502, compilado com **cc65**) [OBS]	CPU `$8000–$FFFF`

Conversão: `endereço_CPU = offset - 0x6000 + 0x8000` para o PRG.

Vetores (em `0xDFFA`): NMI=`$80B4`, RESET=`$8000`, IRQ=`$80EB` (só `RTI`).

O PRG roda **em RAM**: no reset o OS regrava os próprios vetores `$FFFA–$FFFF` (`$8032–$804D`) [OBS]. Logo, o código pode ser patchado à vontade e variáveis podem morar dentro de `$8000–$FFFF` (o próprio OS já faz isso em `$824F–$826E`).

2. Hardware: registradores do cartucho

O cartucho expõe um **banco de registradores indexado**: escreve-se o índice em `$4400` e lê/escreve-se o valor em `$4401` [OBS].

Índice	Uso observado	Confiança
`$09`	bits 0–1 escolhem o que aparece em `$8000–$FFFF`: `01`=RAM do OS, `10`=flash da BIOS (usado só para ler CFI/ID e regravar a flash)	[OBS]
`$80`	banco de PRG durante o carregamento da ROM do jogo	[OBS]
`$81`	`$08` = normal (WRAM em `$6000`). `$09` (bit0=1) = mapeia em `$6000–$7FFF` o banco que contém o **CHR do OS.BIN** (usado para copiar a fonte para a CHR RAM)	[OBS]
`$82`	banco de CHR durante o carregamento	[OBS]
`$83`	último registrador antes de iniciar o jogo (stub em RAM `$8123` → `$0400`)	[OBS]
`$86`, `$88`, `$8B`, `$8C`	configuração de mapper/mirroring/save do jogo (`game_apply_mapper_q $AEE1`)	[INF]
`$0A`	lido na inicialização e guardado em info do dispositivo	[OBS]

(!) **Não mexer** nos stubs de flash (`$847D`, `$84C0`, `$84F3`): eles colocam a flash da BIOS em `$8000` e enviam comandos `AA/55/80/AA/55/10` (apagamento) — risco de brick.

3. Mapa de memória em tempo de execução

Faixa	Uso	Confiança
`$0000–$00FF`	zero page. cc65: `c_sp=$28`, `sreg=$2A`, `regsave=$2C`, `ptr1=$30`, `ptr2=$32`, `ptr3=$34`, `ptr4=$36`, `tmp1=$38`. OS: `$92/$93` ponteiro do texto, `$9A–$9D` ponteiros PPU, `$9E` flag NMI, `$9F` cor do texto (`$FF` normal / `$7F` destaque = +`$80` no tile), `$A0` joypad, `$A2` NMI ocupado	[OBS]
`$0100–$01FF`	pilha do 6502	[OBS]
`$0200–$02FF`	**sem uso** (só zerado no reset) → **ideal para a tabela OAM (sprites)**	[OBS]
`$0300–$04A2`	segmento DATA do cc65 (copiado de `$F9FF` no boot). Tabela de ponteiros de strings de menu em `$040A`	[OBS]
`$04A3–$0573`	BSS do cc65	[OBS]
`$04A9–$04E8`	**tabela de 32 ponteiros copiada de** `$FC00` — API da BIOS (SD/disco)	[OBS]
`$0500–$05FF`	variáveis; também usado como buffer temporário pelos stubs de flash	[OBS]
`$0600–$07FF`	pilha de software do C (cresce para baixo a partir de `$0800`)	[OBS]
`$6000–$7FFF`	WRAM. Heap tipo pilha (`heap_alloc $A41A` / `heap_free $A43F`, ponteiro em `$04E9`). `$6000–$63FF` = **buffer da tela** (espelho do nametable `$2000`), `$6400–$680F` = backup da tela, depois tabelas do diretório e buffers de setor. Após o init o heap vai até ~`$7C1C`; pico transitório ~`$7E1C`. `$7FFF` é lido como flag	[OBS]/[INF]
`$8000–$FBA1`	código + rodata + imagem do DATA	[OBS]
`$FBA2–$FBFF`	94 bytes zerados no arquivo — **livres**	[INF]
`$FC00–$FFF9`	zerado no arquivo, mas **preenchido pela BIOS em tempo de execução** (tabela de API em `$FC00`, info de versão/data em `$FFE8–$FFEF`, provavelmente o driver de SD). **NÃO está livre**	[OBS]
`$FFFA–$FFFF`	vetores	[OBS]

Achado crítico: a área zerada no fim do PRG (`$FBA2–$FFF9`, ~1,1 KB) parece livre, mas `os_init` copia 64 bytes de `$FC00` e lê versões em `$FFE8+`. A BIOS escreve ali antes de iniciar o OS.

Espaço que dá para recuperar para código novo

Onde	Tamanho	Observação
`$FBA2–$FBFF`	94 B	livre
`$81C5–$820F`	75 B	rotina de checksum (`$4800` 2 KB + `$6000` 8 KB) **sem nenhum chamador** — código morto
Strings do "About" (`$F765–$F7E6`)	~130 B	basta reapontar as entradas em `$040A` para uma string vazia

Onde	Tamanho	Observação
CHR tabela 1 (OS.BIN `0x1600-0x1FEF`)	~2,5 KB	banco mapeável em `$6000` com reg `$81`=`$09`. Dá para usar como armazenamento ou até executar código (com um trampolim, pois o heap some enquanto o banco está mapeado) [?]
`$4800-$5FFF` (OS.BIN `0x2800-0x3FFF`)	até 6 KB	a rotina de checksum morta soma `$4800-$4FFF`, sinal de que existe memória ali [?] — **testar**

4. Vídeo (PPU)

- `PPUCTRL` = `$80` (NMI on, fundo e sprites na tabela `$0000`); `$84` durante a atualização em coluna [OBS].
- `PPUMASK` = `$0A` → **fundo ligado, sprites DESLIGADOS** [OBS]. O OS não usa sprites nem `OAMDMA $4014` em lugar nenhum.
- Paleta do fundo (`$825F`, 16 bytes): `0F 10 0F 28 | 0F 0F 0F 0F | 0F 0F 0F 0F | 0F 30 00 28`. O fundo "vazio" é preto (`$0F`), igual ao backdrop.
- `ppu_screen_init_upload_chr $826F` **zera os 8 KB de CHR RAM** e copia a fonte: muda reg `$81` para `$09`, copia `$6000-$7FFF` → PPU `$0000-$1FFF`, volta para `$08`. => **O CHR é RAM e o OS.BIN** `0x0000-0x1FFF` **é recarregado sempre que a tela é reiniciada.**
- Fluxo de texto: tudo é desenhado no buffer `$6000-$63FF` e enviado à PPU por `ppu_upload_nametable_full $82FA` (redesenho total) ou `ppu_refresh_screen_vblank $8326` (128 bytes por vblank), ou `ppu_update_cursor_column $83E6` (só a coluna do cursor + 2 linhas de status).

Conclusão de vídeo: a tabela 1 (`$1000`) e os 64 sprites estão totalmente livres. A capa pode ser desenhada com **sprites 8×16 vindos da tabela 1**, sem tocar na fonte.

5. Navegador de arquivos

- `browser_draw_list $C96B` desenha 21 itens (`$15`) por página.
- Índice selecionado: `$8259/$825A` (16 bits, posição na lista ordenada).
- Tabela de ordenação: ponteiro em `$04FF` (posição → índice cru no diretório).
- Buffer do nome: ponteiro em `$0501` (alocado com 264 bytes; nome longo LFN).
- Obter o nome de uma entrada (sequência exata do OS, `$C9FF`):

```
pusha0(raw_index)          ; JSR $F3D9
pushax(*$0501)             ; JSR $F3DB (buffer de nome)
push0 ; push0              ; JSR $F3D7 ×2
JSR dir_get_entry ($9290)  ; retorna código de erro em A
```

- Linhas na tela: itens nas linhas 2–22 (`y=16..183 px`), nome a partir da coluna 2; comprimento máximo do nome na lista = `$1D` **(29)** → byte em `$CB0E` (`LDA #$1D` em `$CB0D`). Linhas 25–26 mostram o nome completo do selecionado (`$1E`=30 colunas, `$CA5C` e `$CABE`).
- Diretórios usam a fonte de destaque (`$9F=$7F`).
- API de texto: `txt_gotoxy(x,y) $A52E`, `txt_print_xy(str,x,y) $A604`, `txt_print_xy_n(str,x,y,max) $A5D2`, `txt_puts_n(str,max) $A584`, `txt_clear_row(y) $A557`, `txt_println(str) $A808`, `scr_clear_console $A7F8`.

Diretório / FAT (`fat_dir_scan $8DFE`)

- Entradas com atributo **Oculto (`0x02`)** ou **Volume (`0x08`)** são **ignoradas** [OBS]. => arquivos de capa marcados como ocultos não aparecem na lista.
- Limite de **~254 entradas** por pasta [OBS].
- Cada entrada guarda uma **chave de 4 bytes = 4 primeiros caracteres do nome, em minúsculas** [OBS].
- `file_find_by_name $9E1C(dir_cluster32, "NOME8 EXT", ?, &out)` procura por **nome 8.3 exato** (formato de 11 caracteres com espaços, ex. `"MAPROUT BIN"`, `"000 RBF"`) e devolve o cluster/tamanho [OBS]. Usada para `MAPROUT.BIN` (`$BC4E`) e para os núcleos `.RBF` (`rbf_find_core_file $B1AA`). **Ela também chama** `fat_dir_scan`, então também sobrescreve o cache da pasta que o navegador está mostrando. Use só na inicialização.
- `dir_find_by_key4 $ACCE(dir_cluster32, "nome", &out)` acha uma entrada comparando **só essa chave de 4 letras** e, para isso, **re-escaneia o diretório no cache global** (destrói a listagem atual) [OBS]. Não serve para procurar capa por nome do jogo.
- Pastas do sistema (nesta versão clone): `/edfc`, `/edfc/save`, `/edfc/maps`, arquivo `MAPROUT.BIN`. Os clusters são resolvidos em `os_init` (`$BB43-$BC3B`) e guardados em `$0507` (edfc), `$050B` (save), `$050F` (maps) [OBS].

- Abrir arquivo pelo cluster: `file_open_cluster $95C5`; ler setores: `file_read_sectors $9619(dst, n)`. **Se** `dst < $2000` **os dados vão direto para a PPU** (`disk_read_to_ppu $AA41`), senão para a memória da CPU (`$A91B`) [OBS]. => dá para fazer streaming dos tiles da capa do SD para a CHR RAM sem buffer.
- Estado do arquivo aberto: `$0559–$055F` (cluster atual, setor no cluster); `$0541` = setores por cluster. Próximo cluster: `fat_next_cluster_q $C58D` [INF].
- Leitura de setor: `disk_read_sector $A989` → `disk_cmd_sector $A85C`, que escreve um comando (`'R'`, LBA) num buffer e chama a BIOS via ponteiro da tabela `$04A9`. Cache do último LBA em `$8254–$8257` [OBS].

6. Projeto proposto para as capas

6.1 Formato da capa

- **64×96 px** = 8×6 sprites de 8×16 = **48 sprites**, **96 tiles** (1.536 bytes) na tabela 1.
  - 8 sprites por linha de varredura = exatamente o limite do NES (nada mais pode usar sprites ali).
  - Alternativa: 64×112 (56 sprites) ou 64×128 (64 sprites, todos).
- Cores: 4 paletas de sprite × 3 cores, **paleta escolhida por bloco de 8×16** (melhor que os 16×16 dos atributos do fundo). Pixel transparente mostra o preto do fundo => o preto vira uma 13ª cor "de graça".
- Posição: colunas 23–30 (x=184), linhas a partir de y≈24. Reduzir o nome na lista de 29 para ~21 caracteres (patch de 1 byte em `$CB0E`).

6.2 Patches de base

1. `PPUCTRL` com sprites 8×16 (bit 5): `$82EC: 80→A0`, `$841B: 80→A0`, `$BCB1: 80→A0`, `$8400: 84→A4`. Sem efeito enquanto os sprites estiverem desligados.
2. Depois de `browser_draw_list`, um gancho (ex.: trocar o `JSR $83E6` em `$CB78` e o caminho do `$82FA` em `$CB67`) que: carrega a capa → monta OAM em `$0200` → `OAMDMA` → `PPUMASK=$1E`.
3. Qualquer outra tela do OS volta a escrever `PPUMASK=$0A`, então os sprites somem sozinhos na maioria dos menus. Popups desenhados por cima do navegador **[?]** precisam ser verificados (esconder sprites com Y=`$F0`).

6.3 Armazenamento no SD (recomendado)

Como a busca por nome do OS só usa 4 letras e destrói o cache do diretório, a melhor saída é um **pacote único** (como o `.PAK` + catálogo do Mega Drive):

- `/EDFC/COVERS.BIN` (nome 8.3 → `"COVERS BIN"`).
- O cluster do pacote é resolvido **uma vez** no `os_init` com `file_find_by_name $9E1C`, logo depois da busca do `MAPROUT.BIN` (`$BC4E`) e **antes** de o navegador abrir qualquer pasta. Guarde os 4 bytes do cluster numa área livre.
- Cabeçalho: tabela de 256 baldes (pelo byte baixo de um hash32 do nome do arquivo) → setor do balde; cada balde lista `hash32 → setor inicial`. Busca = 2 leituras de setor.
- Registro da capa (4 setores = 2 KB): 16 B de paleta + 48 B de atributos/OAM + 1.536 B de tiles.
- Leitura: abrir pelo cluster (`$95C5`), avançar clusters (`$C58D`) até o setor desejado, `file_read_sectors(dst=$1000, n)` direto na PPU com a renderização desligada por 1–3 frames.
- Ferramenta no PC (Python + Pillow, como nos outros projetos): recorta/redimensiona para 64×96, quantiza para 4 paletas NES × 3 cores por bloco 8×16, gera os tiles e monta o `covr.bin` usando o **mesmo hash** do código 6502.

6.4 Prova de conceito sem SD (primeiro passo, risco quase zero)

Construída sobre o build base do clone (seção 10): `poc_capa_fixa/EDFC/OS.BIN`, gerada por `ferramentas/build_poc.py`. A versão para o v1.26 fica em `poc_capa_fixa_v126/`. A lista de testes está em `poc_capa_fixa/LEIA-ME.md`. **Resultado no hardware (base):** a capa aparece. Na 1ª versão ela aparecia também em pastas e em outras extensões, então foi adicionado um filtro. O OS escreve o nome completo do selecionado nas linhas 25–26 do buffer de tela (`$6320–$635F`, zeradas antes). O gancho só mostra a capa se os 4 últimos caracteres não-nulos, com `|$20`, forem `.nes`; senão grava `PPUMASK=$0A`. Pastas não casam porque não têm a extensão. A capa ficava por cima do **File Menu**. Todo popup/menu redesenha via `ppu_refresh_screen_vblank $8326`, que o laço do navegador (`$D953`, que chama `browser_draw_list` a cada volta) nunca usa. O `JMP $8343` em `$833D` virou `JSR $81DE` (o "no" do filtro, `PPUMASK=$0A`). **Espaço:** a região morta `$81C5–$820F` está usada até `$820C` e o fim do PRG até `$FBFE`. Para a leitura do SD vai ser preciso achar mais espaço (strings do About, banco CHR ou `$4800`, seção 3). Por falta de espaço, a paleta dos sprites foi para o banco CHR (`0x1EF0` → `$7EF0`) e o código foi dividido em `$81C5` (hook + parte A) e o fim livre do PRG (parte B).

Como o OS.BIN `0x1000–0x1FFF` é copiado para a CHR `$1000` sempre que a tela é iniciada, dá para **colocar uma capa fixa direto no OS.BIN** e só escrever ~40 bytes de código (OAM + `OAMDMA` + `PPUMASK=$1E`) em `$FBA2`. Isso valida no aparelho: sprites 8×16, posição, cores e a convivência com o menu, antes de mexer com leitura do SD.

7. Testes pendentes no hardware (diagnóstico)

1. **Quanto de `$FC00–$FFF9` a BIOS ocupa?** Build de diagnóstico que copia `$FC00–$FFFF` para a tela em hex (ou para a WRAM e usa "SRAM To File").
2. `$4800–$5FFF` **é RAM?** Escrever/ler padrões e mostrar o resultado. O OS.BIN `0x2800–0x3FFF` aparece ali (procurar `$AA`)?
3. **Topo do heap da WRAM**: confirmar que `$7E20–$7FFE` fica livre em todos os menus.
4. **Velocidade de leitura de setor** (quantos frames por setor) para dimensionar a capa.
5. O OS é carregado de `/EDFC/OS.BIN` (confirmado pelo pacote). Falta testar se a BIOS aceita um OS.BIN **maior que 56 KB**.
6. **Popups do navegador** ("File Menu" etc.) apagam ou não os sprites.

8. Mapa de funções (resumo)

Runtime do cc65 em `$ED50–$F532` (todas renomeadas com prefixo `cc65_` no projeto). Funções do OS mais relevantes:

Endereço	Nome dado	O que faz
`$8000`	`reset`	init HW, zera RAM, `copydata`/`zerobss`, `c_sp=$0800`, vai para `main`
`$80B4`	`nmi_joypad`	lê joypad 1/2 para `$051D` se `$A2==0`
`$826F`	`ppu_screen_init_upload_chr`	zera CHR, paleta, copia fonte (reg `$81`)
`$82FA` / `$8326` / `$83E6`	upload de tela	total / progressivo / só cursor
`$8393`	`ppu_wait_vblank`	
`$83C5`	`scr_clear_buffer`	zera `$6000–$63FF`
`$8DFE`	`fat_dir_scan`	lê diretório para o cache (chaves, flags, ordenação)
`$9290`	`dir_get_entry`	nome/infos da entrada N
`$95C5` / `$9619`	abrir por cluster / ler setores	PPU ou CPU
`$A41A` / `$A43F`	`heap_alloc` / `heap_free`	heap em pilha na WRAM
`$A85C` / `$A989`	comando de disco / ler setor	via API da BIOS
`$ACCE`	`dir_find_by_key4`	busca por 4 letras
`$ADA3`	`file_open_entry`	abre arquivo da pasta atual
`$B280`	`game_load_rom`	lê cabeçalho iNES, PRG (reg `$80`) e CHR (reg `$82`)
`$B7EF`	`os_init`	heap, tabela da BIOS, pastas edfc/save/maps
`$C96B`	`browser_draw_list`	**ponto do gancho das capas**
`$ED50`	`main`	

Nomes terminados em `_q` são palpites com evidência parcial.

9. Pacote completo da firmware (nesos-v1.26)

Cópia em `firmware_original/EDFC/` (hashes em `firmware_original/SHA1SUMS.txt`). O `OS.BIN` do pacote é **idêntico** ao analisado (SHA-1 `8a5cd565…defce`).

Estrutura no SD:

```
/EDFC/OS.BIN        OS (56 KB) — o que vamos modificar
/EDFC/MAPROUT.BIN   256 bytes: mapper iNES → número do núcleo FPGA (FF = sem suporte)
/EDFC/MAPS/NNN.RBF  núcleos FPGA (bitstream Altera "raw binary"), um por família de mapper
/EDFC/SAVE/         saves
/EDFC/mappers.png   tabela de compatibilidade (177 mappers suportados, versão 4.4.2021)
```

`MAPROUT.BIN` decodificado:

Núcleo	Mappers iNES
000	0, 2, 3, 7, 16, 18, 32, 33, 36, 38, 48, 66, 71, 87, 140, 219
001	11, 13, 34, 65, 67, 68, 70, 72, 76–78, 80, 82, 86, 88, 89, 92–97, 144, 152, 154, 180, 184, 185, 206, 232
002	15, 28, 31, 40–42, 57, 58, 61, 91, 99, 101, 107, 112, 164, 168, 178, 188, 193, 200–203, 212, 227, 231, 234, 240–242, 246
003	79, 90, 113, 133, 137–139, 141, 146–148, 150, 211, 243
004	1, 4, 9, 10, 220 (MMC1/MMC3/MMC2/MMC4)
005	5 (MMC5)
006	12, 47, 64, 74, 115, 118, 119, 158, 182, 189, 191, 196, 205, 245
007	30, 111, 190, 207

Núcleo	Mappers iNES
019	19 (Namco 163) · 021: 21–26, 73, 75, 151 (VRC) · 027: 27, 50, 106, …
037	35, 37, 44, 45, 49, 51, 52, 60, 105, … · 056: 56, 103, 132, …
069	69 (Sunsoft 5B) · 085: 85 (VRC7) · 254: 254

- O OS monta o nome `"NNN RBF"` e procura em `/EDFC/MAPS` (`rbf_find_core_file $B1AA`, chamado por `file_open_entry $ADA3`). `$8252` guarda o núcleo carregado da última vez.
- `255.RBF` **não aparece no MAPROUT** e o OS nunca pede o núcleo 255. Provavelmente é o núcleo do próprio menu, carregado pela BIOS [?]. Os registradores `$4400/$4401` que o OS usa seriam definidos por ele. **Não precisa mexer nos** `.RBF` **para as capas**: elas vivem só no `OS.BIN` + um arquivo de pacote.
- Os `.RBF` são bitstreams de FPGA (cabeçalho `FF…FF 6A DE FF 7F`); não há código 6502 dentro deles.

Instalação da custom firmware: basta trocar `/EDFC/OS.BIN` no SD (e adicionar o `/EDFC/COVERS.BIN`). Para voltar ao original, copie de volta o `OS.BIN` de `firmware_original/`.

10. Build BASE do clone (o que vale para a custom firmware)

O OS que roda no cartucho clone é **outro build**: pasta `EDFC-20261023T010636Z-1-001`, cópia em `firmware_base/EDFC/`, SHA-1 `dbea89d33eacd7ae72ee24d2652727b0cb5740ae`. As seções 1–8 foram escritas sobre o v1.26 do krikzz e continuam válidas na lógica, mas **os endereços abaixo de** `$84F5` **em diante mudam**. O que muda no build base:

- O CHR (`0x0000–0x5FFF`) é idêntico; só o PRG muda (29.482 bytes).
- `$8000–$84C0` não mudou de endereço (reset, NMI, PPU, stubs de flash). A rotina morta `$81C5–$820F` é a mesma.
- Código C deslocado: −60 bytes de `$84F5` até ~`$AA05`, −53 em `file_open_entry`, +56 em `game_load_rom` e +28 a partir de ~`$C337` (inclui `browser_draw_list $C987`).
- DATA `$0300–$049A` (carregado de `$FA39`), BSS a partir de `$049B`. As **variáveis em RAM recuaram 8 bytes** em relação ao v1.26, então nomes de variáveis em `analise/` não valem aqui.
- A tabela da BIOS continua vindo de `$FC00` (`LDX #$FC` em `$B6FD`).
- **Espaço livre no fim: só** `$FBD4–$FBFF` **(44 bytes).**
- **Verificação de clone:** a rotina `$B255` recebe um código. Se for diferente de `$0F`, ela pinta o fundo, escreve `ERROR 5FA1.Clone detected!` e trava em `$B252` (`JMP $B252`). Não foi tocada.
- `os_init`, `main`, `copydata`, `zerobss`, `dir_find_by_key4`, `rbf_find_core_file`, `file_menu_q` e outras mudaram de corpo; não foram mapeadas automaticamente.

Arquivos:

- `analise_base/ghidra_proj/EDFC_OS_base.gpr`: projeto Ghidra deste build.
- `analise_base/ghidra_out/`: listagem, decompilação e referências.
- `analise_base/mapa_funcoes.txt`: endereço v1.26 → base para as 169 funções casadas.
- `ferramentas/map_funcs.py`: o casador de funções.
- `ferramentas/ghidra_scripts/nomes_base.txt`: os nomes gerados para o Ghidra.

Pontos de patch da PoC no build base: `$CB8B` (`PPUMASK` do redesenho completo), `$CB94` (`JSR $83E6` do cursor), `$CB29` (tamanho do nome), `PPUCTRL` em `$82EB/$841A/$BB9B/$83FF`.

11. Resultado do diagnóstico no hardware (DIAG 1, foto em `imagens/1.jpeg`)

Teste	Resultado	Conclusão
RW `$4800/$5000/$5FFF`	`5AA5 5050 5F5F`	só `$4800` responde como memória. `$5000+` é barramento aberto (devolve o byte alto do endereço)
`$4800–$4807`	`00 02 20 00 08 04 40 00`	há dados ali. Provavelmente RAM de 2 KB (`$4800–$4FFF`, a faixa que o checksum morto somava) usada pela BIOS/FPGA → **não usar**
`$AA` em `$4800–$5FFF`	`0001`	a BIOS **não** carrega `OS.BIN 0x2000–0x5FFF`; esse trecho do arquivo é só preenchimento
Tabela `$FC00` (32 palavras)	`0080 0081 0082 0083 0084 0086 0088 0089 008A`, depois `FE74 FEB8 FEEB FEFD FF05 FC53 FC64 FC77 FC7F FD11 FD4A FD8B FDD8 FE31 FE53 FE5B FF27 FC87 FE96 FF34 FF0D FFA7`	as 9 primeiras são **índices de registradores** do cartucho; as 23 seguintes são rotinas da BIOS em `$FC53–$FFA7`
Último não-zero `$FC00–$FFF9`	`$FFEF`	a BIOS ocupa toda a faixa (só `$62` zeros espalhados) → sem espaço livre ali

Teste	Resultado	Conclusão
Último não-zero `$7C00–$7FFE`	`$7C84` (no menu principal)	topo do heap ~`$7C84`. Picos de menus podem somar ~`$200`. Só `$7F00–$7FFE` é razoavelmente seguro

Espaço disponível para o carregador de capas:

- **Banco CHR** (reg `$81`=`$09`, `$6000–$7FFF`): RAM de 8 KB própria. A tabela 1 tem ~2,3 KB sem uso (`0x1600–0x1EEF`). Dá para guardar dados e até código, mas enquanto ele está mapeado a WRAM (heap, buffer de tela, buffers do FAT) some.
- **Tela About** (`$D2B9–$D329`, 113 B) e suas strings de créditos (~130 B em `$F7A0–$F821`), se o About for sacrificado.
- `$7F00–$7FFE` da WRAM (~250 B), com risco.

12. Firmware com capas do SD (`capas_sd/`, `ferramentas/build_capas.py`)

Espaço usado (com About e Device Info removidos, decisão do usuário):

Área	Uso
`$81C5–$81FF`	`hook_cursor`, `draw_hook` (filtro `.nes`), `no_cover`/`hide`, `idle_hook`, tabela `.nes`
`$D0C6–$D308` (Device Info + About)	`info_screen`, `init_hook`, `crc_part`, `load_cover`
`$F79B–$F839` (textos do About)	`lba_read`, `setP`, `cmp3`, `scroll0`, variáveis, strings
`$FBD4–$FBE6`	`cover_show` (vblank, OAMDMA `$0200`, `PPUMASK=$1E`)

Ganchos: `$BB4A` (busca do MAPROUT no `os_init` → `init_hook`), `$D9EB` (espera ociosa do laço do navegador `$D953` → `idle_hook`), `$CCFC` (About → `info_screen`), `$833D` (menus → `hide`), mais os da PoC (`$CB8B`, `$CB94`, `PPUCTRL`, `$CB29`).

Rotinas do OS usadas (build base): `file_find_by_name $9DE0` (u32 dir, nome 8.3, u32* tam, u32* out; a função remove 10 bytes da pilha do C), `fat_cluster_to_lba $96B3`, `fat_read_sector_cached $C53F` (u32 lba → `*($04E9)`, cache em `$0529–$052C`), `ppu_wait_vblank $8393`, espera ociosa `$AB25`, joypad em `$0515`, cluster da pasta `/edfc` em `$04FF–$0502`. Zero page própria: só os temporários do cc65 `$30–$35` (a BIOS usa `$80–$8A`).

Formato do `COVERS.BIN`: setores 0–255 = baldes do índice (`crc & $FF`), entradas `[crc>>8, crc>>16, crc>>24, idx_lo, idx_hi]`, fim = `idx_hi $FF`. Capa `i` em `256+4i`: OAM[256] + paleta[16] + verificação[3] e depois 1.536 bytes de tiles. CRC = CRC-32 do zlib sem o XOR final, sobre o nome exibido nas linhas 25–26 (≤ 60 caracteres) com cada byte `| $20`.

Testes no PC (`ferramentas/teste_capas_emu.py`, emulador 6502 + rotinas do OS simuladas): 32 verificações (CRC, tiles, paleta, OAM, cache, sem capa, pasta/`.fds`, rolagem, arquivo fragmentado, sem `COVERS.BIN`). Todas as 192 capas sintéticas de `001-200` e `super mario` carregam.

Resultado no hardware (capas_sd, OS.BIN SHA-1 `7c9f4886…`): `COVERS.BIN: OK` na tela de info. Cada jogo mostra a própria capa (cor, número e nome). Sem tremer nem lixo durante a carga, e a capa aparece rápido. Tempo no vblank: o NMI do OS gasta ~315 ciclos; cada pedaço de 128 bytes (laço desenrolado, ~11,5 ciclos/byte) mais a montagem dá ~1.850 de 2.273 ciclos (NTSC).

14. EXPERIMENTO v2: sprites + camada de fundo (`experimento_v2/`, `ferramentas/build_capas_v2.py`)

> Experimental. A referência estável é a v1 (favorita). Os formatos de `COVERS.BIN` v1 e v2 são incompatíveis.

Premissas verificadas no OS base: o OS nunca escreve atributos (buffer zerado → tudo na paleta 0); as paletas de fundo 1–3 não são usadas (`$825F`: `0F 10 0F 28 | 0F.. | 0F.. | 0F 30 00 28`; só `$A800` grava `$3F00/$3F01` antes de iniciar jogo); os tiles `$00–$1F` e `$80–$9F` da tabela 0 são sólidos na cor 2 (vazios): **63 slots livres** (`$01–$1F`, `$80–$9F`; o `$00` é a célula vazia do texto).

Geometria: capa em x=176/y=32 (colunas 22–29, linhas 4–15), alinhada à grade de atributos; nome na lista com 20 colunas (`$CB2A`=`$14`). Os atributos `$63CD–$63DF` (19 bytes contíguos) cobrem a capa; os bytes de texto dentro deles ficam 0.

Firmware (áreas: `$81C5` 67/75, `$FBD4` 38/44, `$D0C6` 606/612, `$F79B` 250/251):

- `load_cover`: setor = 256 + 8·idx. Metadados: OAM[256] + paletas `$3F04–$3F1F`[28] + verificação[3] + NT[96 em 288] + atributos[19 em 384]. NT e atributos vão para o **buffer de tela** do OS; os tiles (3 setores de sprite → `$1000`, 1 → `$0000`, 1 → `$0800`) vão em pedaços de 128 B por vblank; no fim, `upload_bg` envia NT e atributos do buffer para a PPU (6 linhas por vblank + atributos).

- `hide_draw` (troca de item): se o fundo estiver visível, `bg_clear` (zera só tiles com `t & $60 == 0` e os atributos) + `upload_bg` e `has=0` (voltar ao jogo recarrega).
- `hide` (`$833D`, dentro do refresh dos menus): só `bg_clear` no buffer; o texto do popup é preservado e o OS restaura a capa com o backup da tela.
- `mask_entry` (redesenho completo, `$CB8B`): invalida o CRC atual → recarrega.
- Device Info/About → `RTS`.

Conversor (`ferramentas/conv_v2.py`): 3 paletas de fundo (blocos 16×16) + 4 de sprite (8×16) por busca local (distância Lab com peso no tom); decomposição por pixel (sprite tem prioridade; pixel do fundo sob o sprite é "tanto faz"); tiles do fundo agrupados (compatíveis sem perda, depois fusão do par de menor custo) até 63. Mario 3: 82 tiles → 63 com perda pequena; 9 → 14 cores em relação à v1.

Teste no PC: `ferramentas/teste_capas_v2_emu.py`, tudo OK (OAM, paletas, tiles, NT/atributos no buffer e na PPU, popup preservado, limpeza ao sair, recarga ao voltar, troca de página, sem capa, `.fds`, fragmentado, sem arquivo). **Teste 1 no hardware (foto `imagens/2.jpeg`):** a capa melhorou bastante, mas houve dois problemas:

1. **A barra de baixo (nome do jogo) ficou ciano.** A premissa "paletas 1–3 livres" estava **errada para a paleta 3**: ela (`0F 30 00 28`) é a da barra de baixo. O OS deve definir esses atributos fora do buffer ou na inicialização; a busca por escritas diretas não encontrou. As paletas 1 e 2 (pretas) são as livres. Correção no conversor: só 2 paletas de fundo (atributos 1–2), e a gravação de `$3F04–$3F1F` regrava `$3F0C–$3F0F` com `0F 30 00 28`.
2. **A capa quebrava depois do File Menu.** O `hide` do `$833D` roda em todo refresh do OS, inclusive o que restaura a lista depois do menu. Ele limpava o fundo restaurado, mas `has` continuava 1 e só os sprites voltavam. Correção: `hide` também zera `has` e `bgshown`, e a capa é recarregada sozinha ao voltar (~0,5 s). Com 2 paletas de fundo, o Mario 3 ficou com 16 cores (v1: 9). Teste no PC: 86 verificações OK. Novo `OS.BIN` v2: SHA-1 `11ae315973cf0661e072aecd5b0933536402cff1`.

Teste 2 no hardware (foto `imagens/3.png`): a barra e a volta do File Menu ficaram OK, mas faltava parte da **caixa cinza do File Menu**. O popup também usa a **paleta 3 via atributos**, e o `bg_clear` zerava os 19 bytes de atributo inteiros (incluindo a caixa). Correção: cada campo de 2 bits com 3 é mantido e os outros (paletas 0–2 da capa) viram 0 (`m = b & (b>>1) & $55; b = m | m<<1`). Para caber: `hide` foi para `$81C5`, `setP` para `$D0C6`, `base` é zerado num laço e saiu um `CLC` redundante (carry sempre 0 com idx < 8160). Áreas: `$81C5` 73/75, `$FBD4` 38/44, `$D0C6` 612/612, `$F79B` 247/251. Teste no PC: 87 verificações OK. `OS.BIN` v2: SHA-1 `8495d728d789868ec21ad17f7ef2b953a559e873`.

Teste 3 no hardware: caixa cinza inteira e a capa volta ao fechar o menu. Mas, ao **abrir** o menu, o fundo da capa "quebrava" por alguns quadros antes de sumir (foto `imagens/4.jpeg`: blocos branco/rosa). Causa: `ppu_refresh_screen_vblank` envia o buffer em 8 pedaços de 128 B (um por quadro) e os atributos vão **por último**. Os tiles da capa já chegavam vazios (tile `$00` = cor 2) enquanto os atributos antigos (paletas 1–2) continuavam → blocos sólidos nas cores da capa. Além disso, o `bg_clear` gastava o vblank que o OS tinha reservado para o 1º pedaço. Correção: `hide` agora (1) pinta `$3F04–$3F0B` de preto no próprio vblank (o fundo da capa some no mesmo quadro que os sprites), (2) refaz a rolagem, (3) limpa o buffer e (4) espera o próximo vblank antes de devolver ao OS. As paletas voltam ao carregar a próxima capa. Espaço: os textos da Device Info (`$F75F–$F79A`, 60 B, só usados pela tela removida; o `8D F7` em `$B7BB` é um `STA $04F7`, coincidência) foram juntados à área do About: `$F75F` 279/311. Também livre (não usado): `cc65_decaxy` `$EF64–$EF6C` (9 B, só a Device Info chamava). A `$A680` **não** está livre (é chamada por código em `$A697+` que o Ghidra não desmontou). Teste no PC: 89 verificações OK. `OS.BIN` v2: SHA-1 `c0a74aeab86b2ad3594b062535e4ef88cb866818`. **Teste 4 no hardware: OK** (a capa some de uma vez ao abrir o File Menu).

Paleta 0 (texto: cinza `$10`, preto, amarelo `$28`): tirar o amarelo do menu não libera cor (a paleta 0 é de todo o texto). Em vez disso, o conversor pode usar a paleta 0 **como ela é** em blocos 16×16 da capa (atributo 0), sem custo nenhum. A paleta 3 não pode, porque é ela que protege a caixa do menu na limpeza. No Mario 3 o otimizador não escolheu a paleta 0 (prefere `$39` ao `$28` para aquele amarelo).

16. Palette Lab local (`ferramentas/palette_lab/`)

Reimplementação em Python + tkinter de nesdevtools.com/palette-lab. Levantamento feito com o REA (CDP: `list_browser_targets` + `analyze_web_bundle`; 1 script inline de 47 KB, SHA-256 `204675e8…`) em `palette_lab_analise/FUNCIONALIDADES.md`. Funções do site + modo **capa EDFC** (64×96, bloco 8×16) com "Salvar capa para o COVERS…" (`<ROM>.json`) e "Gerar COVERS.BIN…". O `gerar_covers.py` aceita esses .json (prioridade sobre PNG; v1 sem reconversão via `capa_edfc.record_v1`). Na capa do Mario 3, "fidelidade de cor" 75% deu o melhor resultado (amarelo, vermelho, azul e tom de pele), então vira o padrão do modo capa. Instalado no Python do usuário: Pillow 13.0 (`pip install --user pillow`).

17. POC v3: alternância de quadros (`experimento_v3/`, `ferramentas/build_capas_v3.py`, `conv_v3.py`)

Simulação (`ferramentas/sim_flicker.py`, `experimento_flicker/`), erro médio ΔE no Mario 3: v1 29,6 · v2 26,1 · alternância λ=0,05 15,4 (91 cores percebidas, tremido 47) · λ=0,15 19,3 (tremido 24) · λ=0,20 21,7 (tremido 13). Com a mesma paleta por bloco nos dois quadros (necessário para a firmware): 15,7. O usuário pediu a POC com fidelidade máxima (λ=0,05).

Firmware (a partir da v2 estável):

- `$80E6` (fim do NMI: `LDA $2002/PLA/RTI`) → `JMP nmi_flk`. Com `flk=1` e `$A2=0`: grava `$3F04–$3F1F` a partir de `PALTAB+frame` (WRAM `$7F00`, A=0..27, B=28..55), faz o OAMDMA de `$0200`, `scroll0`, e depois `tile EOR $80` nos 48 sprites e `frame EOR 28`. Tiles A = 1..95 (`$1000`), B = 129..223 (`$1800`).
- `flk=0` em: `load_cover` (também `frame=0`), `hide_draw`, `h_inv` (menus) e no wrapper do `$82FA` (`$CB83`). `cover_show` faz `flk=1`.
- `COVERS.BIN` v3: setor = 256 + idx·16. Metadados: OAM A, paletas A [256], B [284], verificação [312], NT [320], atributos [416]. Setores 1–3 sprites A, 4–6 sprites B, 7–8 fundo. Capas sem PNG/json saem com A=B.
- Espaço: `$A680–$A706` livre (135 B; `$A697/$A6B9/$A6DC` sem nenhuma chamada, `$A680` só chamada por elas e pelo About), mais `$F753–$F75E` (título da Device Info). `cc65 ldeaxi` = `$F14C`.
- Orçamento do NMI: o do OS (~220 ciclos) + paletas (~420) + DMA (514) + scroll (~30) ≈ ciclo 1.250 de 2.273. A troca de tiles (~1.150 ciclos) roda fora do vblank. Teste no PC (`teste_capas_v3_emu.py`): tudo OK, incluindo a média dos quadros reconstruídos a partir da PPU = prévia (6144/6144 px). `OS.BIN` v3 SHA-1 `5b23f07edb23ed5cac2bf8c0fcaa0f022eabea6c`. Hardware: *pendente*.

POCs extras (mesmo `OS.BIN`, só muda o `COVERS.BIN`; `gerar_covers.py --v3 --lambda X`):

Pasta	λ	ΔE Mario 3 (sim.)	Tremido	`COVERS.BIN` SHA-1
`experimento_v3/`	0,05 (máxima)	15,4	47	`c79390f0d9b1dbac987fd523427d600ab72368d6`
`experimento_v3_intermediaria/`	0,15	19,3	24	`5c9e07cceaa143692725e29f985c68a44ac4f71f`
`experimento_v3_suave/`	0,20	21,7	13	`1a827fe80dd0c01183e2fc9baa176e0d604ed123`

`teste_capas_v3_emu.py <pasta>` e `visualizador_v3.py <pasta>` aceitam a pasta; os três passam no teste (TUDO OK).

Página de ajuste (`ferramentas/sim_flicker_barra.py` → `experimento_flicker/ajuste_flicker.html`): barra com < e > de λ=0,02 a 0,25 + v2. Acima de λ≈0,22 o tremido quase some e o ΔE volta para perto do da v2. **Escolha do usuário (23/10/2026): λ = 0,15 (nível 6/15, ΔE 19,3, tremido 25,4) = POC intermediária.** Foi o padrão de `gerar_covers.py --v3` até o teste no console; agora o padrão é λ=0,22. **Hardware (23/10/2026): a intermediária (λ=0,15) funcionou no console, mas flicou muito.** O usuário pediu a POC com λ=0,22 (nível 11/15, ΔE 24,2, tremido 3,5): `experimento_v3_lambda022/`, `COVERS.BIN` SHA-1 `dee4037b704dc6b86e9ce6cf1146d5f715c1c3a8` (mesmo `OS.BIN`). Emulador: TUDO OK. Hardware: *pendente*.

Gerador de firmware com teste de flicker (`ferramentas/gerador_firmware_capas.py`)

Pedido do usuário: cada TV reage de um jeito, então a pessoa escolhe o nível no teste embutido e gera a firmware. Atalho: `Gerador de firmware (capas + flicker).bat`, na raiz (abre com `pythonw`, sem console).

- Tela 1: pasta das capas (PNG/.json), pastas de ROMs, "capa de teste" para as ROMs sem imagem, pasta de saída (padrão `firmware_gerada/`).
- Tela 2: a barra `< >` do teste (14 λ de 0,02 a 0,25 + v2), tudo dentro do programa, sem navegador (pedido do usuário). A alternância é sincronizada com o vsync do Windows: `DwmFlush` espera a atualização da tela e `DwmGetCompositionTimingInfo` dá o contador e os Hz. Num monitor de 60 Hz, o quadro mostrado é a paridade do contador; em 120/144 Hz, é o tempo×60. Medido: 60 trocas/s, 0 vsync perdido. Os níveis são calculados em paralelo (`ProcessPoolExecutor`) e ficam em cache (`ferramentas/cache_flicker/`, chave = SHA-1 dos pixels + λ).
- Menu lateral "Capas": todas as imagens da pasta das capas (PNG/.json), com miniatura e busca. Clicar numa imagem mostra como ela fica no NES: calcula primeiro o nível atual e a v2, depois os outros. Os pedidos da imagem anterior que ainda não começaram são cancelados.
- Correção: a área da capa no NES tem tamanho fixo (quadro preto enquanto calcula). Antes, o texto "calculando…" deixava a área com 31×23 px e a capa parecia parada. O teste agora confere por captura de tela (2 imagens alternando).
- Paletas, como no Palette Lab: o painel "Paletas deste nível" mostra os quadros A e B, com fundo 1–2 e sprites 0–3, 3 cores cada. O fundo 0 (texto) e o 3 (barra) são do OS e ficam fixos. Clicar num espaço abre o seletor das 64 cores do NES (proibidas marcadas com ·; o botão "Original" volta a cor). O conta-gotas na capa mostra o espaço de A e de B

do pixel e permite trocar nos dois quadros ou só em um. Também há Desfazer e Restaurar originais. As edições valem por capa e por nível (as paletas de cada λ são outras) e são salvas em `<capas>/paletas_editadas/<nome>.L0.220.json` (ou `.v2.json`), com a chave SHA-1 dos pixels; se a imagem mudar, a edição é ignorada com aviso. `gerar_covers.py --paletas` (padrão `<capas>/paletas_editadas`) as aplica em `record_v3`/`record_v2`: só os bytes de paleta mudam, os tiles não. O cache dos níveis passou à versão 2, que guarda a estrutura (bg, bg_assign, sp_assign, spr e paletas de A/B). `sb.aplicar()` refaz quadros, média, ΔE e tremido. Teste: `teste_paletas_editadas.py` (TUDO OK) e `teste_gerador_gui.py`, que também testa o painel, o conta-gotas e a geração com a cor editada (TUDO OK).

- Pintar pontos (24/10/2026). O usuário relatou que só conseguia editar o quadro B. Causa: no Mario 3 (λ=0,22), em 35 pontos o quadro A mostra o fundo fixo $0F (sprite transparente sobre um tile de fundo com valor 0), enquanto o B usa o Sprite 1 ($0C). O conta-gotas só oferecia o B. Agora, onde um quadro tem cor fixa ($0F ou texto do OS), o seletor oferece pintar o ponto na camada de sprite daquele quadro, com uma das 3 cores do sprite do bloco, só nele ou em "os N pontos iguais". Cada bloco 8x16 tem o seu tile em cada quadro, então isso só muda os tiles daquele quadro. Os pontos ficam em `pix` no JSON (`{"A": [[x, y, v], ...], "B": ...}`) e são aplicados por `conv_v3.com_pontos` (em `record_v3`, `record_v2` e `sb.aplicar`). Testes: TUDO OK (no COVERS só mudam os bytes $0200–$07FF do registro, os tiles de sprite A).
- Nitidez (24/10/2026, pedido do usuário). Checkbox "Nitidez (nesta capa)": `build_poc.carregar_capa(path, True)` faz o mesmo recorte central, mas reduz com Lanczos e aplica um UnsharpMask (raio 1,0, 70%, limiar 1). Sem nitidez, o caminho continua o antigo (`png_read` + `resize` por média), com resultados idênticos. A escolha é por capa, em `<capas>/capas_config.json` (`{"nitidez": [arquivos]}`), que o `gerar_covers.py` lê. As capas nítidas aparecem com * no menu e as paletas editadas delas vão para `<nome>.L0.220.nitido.json`. O config é ignorado na lista de capas e de projetos .json. Teste: com o Mario 3 nítido, o COVERS sai de `record_v3(carregar_capa(..., True))`. Desligando, volta a `dee4037b…`. TUDO OK.
- Força da nitidez por capa (26/10/2026, pedido do usuário). A barra 10–200% (de 5 em 5) fica ao lado do checkbox e é travada quando a nitidez está desligada. `build_poc.resize_nitido(..., forca)` usa UnsharpMask com `percent=forca`. `carregar_capa(path, nitidez)` aceita False, True (70) ou um número. O config ganhou `"nitidez_forca": {arquivo: %}` (70 = padrão, não é gravado). Novas funções: `conv_v3.forca_de`, `nitidez_efetiva` (0 ou a força), `definir_forca`. As paletas editadas usam o sufixo `.nitido` (70%) ou `.nitidoNN`, e o log mostra "nitidez: capa (NN%)". Com 70%, tudo idêntico a antes (caches, edições, COVERS). Teste: a barra em 120% salva, recalcula e gera = `record_v3(carregar_capa(..., 120))`; voltar a 70% devolve a imagem anterior. 77 verificações TUDO OK. Comparação visual: `debug/comparacao_forca_nitidez.png`. Depois foram acrescentados botões `< >` (5% por clique), que aplicam meio segundo depois do último clique.
- Ajustes de interface (26/10/2026, pedidos do usuário): botão "(atualizar) Atualizar" na lista de capas (`atualizar_capas`: procura de novo e mostra "· N nova(s)"). Botões mais claros: `BOTAO = '#54473b'`, borda `#7a6a58`, hover `#6b5c4c`. A opção "Gerar capa de teste" saiu: a ferramenta não passa mais `--sinteticas` (o CLI continua aceitando) e o `gerador_config.json` não guarda mais essa opção. "v2 estável" passou a se chamar "Capa sem Flicker" nos textos. O teste compara com uma referência do gerador de linha de comando sem capas de teste; o registro do Mario 3 continua igual ao da POC λ 0,22. 81 verificações TUDO OK.
- Ícones na lista de capas (26/10/2026): os símbolos de texto */(editar) foram trocados por ícones desenhados com suavização (`icone()`): nitidez = brilho branco de lados curvos (astroide) num círculo dourado; cores editadas = paleta de pintor com pontos coloridos. Ficam numa coluna ao lado da miniatura (`imagem_item`, guardada por capa/estado). As marcas ficam em `marcas_capa[nome] = (nitidez, editada)`. A legenda fica no rodapé da lista, ao lado do total ("13 de 13 imagens · * nitidez · (cores) cores editadas"). A janela abre no topo da área útil da tela (`posicionar_janela`, SPI_GETWORKAREA), sem passar da barra de tarefas; antes, o Windows a abria em y=79 e o rodapé ficava escondido.
- Título e créditos (26/10/2026): a janela se chama "NES N8 Everdrive Covers editor - TavinhoGames" (`TITULO`). O botão "Créditos" (seção 3) abre uma janela com o autor e as redes do usuário (GitHub Guga-Tavinho, Instagram tavinho.games, YouTube @tavinho-games), com links clicáveis (`webbrowser.open`) e os logos desenhados pela ferramenta (`logo()`, simplificados, sem baixar nada). 81 verificações TUDO OK. Na 1ª rodada, 2 verificações que dependem da janela visível falharam, provavelmente porque a janela do teste foi coberta; na 2ª, TUDO OK. Depois: o botão foi para um cabeçalho no topo (`montar_cabecalho`: nome + "* Créditos" em rosa, estilo `Creditos.TButton` `#e0457b`), e o subtítulo da janela de créditos saiu. Para a janela continuar cabendo na área útil (1040 px), os espaçamentos foram apertados e a lista de ROMs passou a ter 2 linhas: a altura pedida foi de 1036 para 988. 81 verificações TUDO OK.
- Barra de porcentagem (25/10/2026). `ferramentas/progresso.py` cuida das etapas e dos avanços; o callback só é chamado quando o % inteiro muda. Pontos instrumentados: `comparar_capas.optimize`, `conv_v2.convert` (pesos medidos: fundo 0–11%, sprites 11–34%, fusão de tiles 34–100%, quadrática), `optimize_frame` (1ª rodada 15–75%) e `flicker_sim` (conversão 0–54%, B 54–75%, A 75–89%, B 89–100%). Medido: 50% do valor chega a 49–53% do

tempo. Os processos mandam (geração, nível, fração) por uma `multiprocessing.Queue` (`initializer=sb.iniciar_worker`). A janela mostra "calculando… N%" com uma barra embaixo da capa no NES e da v2, e o total no status. Os resultados não mudam (POC `dee4037b…`).

- Nitidez marcada por padrão: `NITIDEZ_PADRAO = True`. O `capas_config.json` virou `{"nitidez": {arquivo: bool}}` (a lista antiga é convertida). Capas sem escolha usam o padrão. `gerar_covers.py --nitidez sim|nao` define o padrão (CLI: `nao`; a ferramenta passa `sim`). Migração: capas sem escolha que já tinham cores editadas sem nitidez ficam com `false` (foi o caso do Mario 3).
- Checkbox "Flicker" ao lado da barra (25/10/2026). Desmarcado, vai para a v2 estável (`OS_v2.BIN` + `--v2`, sem alternância). Marcado de novo, volta ao último nível com flicker. A barra tem só os 14 níveis com flicker (a v2 saiu dela). Sem flicker, a barra e os botões < > ficam travados (pedido do usuário). O botão GERAR mostra "nível N · com flicker · λ" ou "SEM flicker · v2".
- Nome curto 8.3 no SD (25/10/2026). ROMs como `contra.nes` (todas minúsculas e cabendo em 8.3) ficam no FAT sem nome longo. O `dir_get_entry` (lido no código v1.26) então copia o nome curto cru, com 11 caracteres e sem ponto (`CONTRA NES`): o filtro .nes e o CRC não batem e a capa não aparece. Contorno: dar à ROM um nome que obrigue o nome longo (`Contra.nes`, maiúsculas e minúsculas misturadas). O usuário confirmou que funcionou. Correção na firmware (normalizar o nome 8.3) ainda pendente/opcional.
- Flicker por capa (25/10/2026, pedido do usuário). A firmware v3 só alterna os quadros A/B de cada registro, então cada capa pode ter o seu λ, ou nenhum (A = B, igual à v2: `record_v3(flicker=False)` usa `conv_v2.convert` + `com_paletas`). No `capas_config.json`, `"flicker": {arquivo: λ | "sem"}` e `"flicker_padrao": λ | "sem"` (sem chave = `PADRAO` 0,22). A barra, o < > e o checkbox valem para a capa selecionada e salvam o ajuste dela (`ir_usuario`; `ir` sozinho, chamado pelo programa, não salva). Botões "Definir como padrão" (o nível desta capa vira o padrão e ela deixa de ter ajuste próprio) e "Usar o padrão nesta capa". O menu lateral mostra "· λ 0.150" ou "· sem flicker" nas capas ajustadas. Na geração: `gerar_covers.py --v3 --flicker-por-capa --lambda <padrão>`. Se nenhuma capa tiver flicker, usa `OS_v2.BIN` + `--v2`. O log mostra "flicker: <capa> · λ …/sem". Teste: Mario 3 sem flicker (com as outras no padrão) gravado com A = B e `OS.BIN` v3; Mario 3 em λ 0,15 idêntico ao da POC intermediária; padrão "sem" → v2. TUDO OK.
- Problema (25/10/2026): "só apareceu a capa do Mario". O COVERS tinha 192 entradas (Mario + capas de teste de `001-200`). A ferramenta não guardava as pastas de ROMs, e cada vez que reabria voltava às pastas de exemplo. `Contra.nes` e as outras do `F:\roms` não estavam na lista, então as imagens delas saíram como "sem ROM correspondente". Correções: `ferramentas/gerador_config.json` guarda a pasta das capas, as pastas de ROMs, a capa de teste e a saída (`ler_prefs`/`salvar_prefs`; o teste usa outro arquivo); a mensagem final lista as imagens que ficaram de fora ("ATENÇÃO"); as prévias antigas são apagadas antes de gerar. Firmware refeita para o usuário com `F:\roms` (COVERS `27db6884…`, 4 capas; Magaman.nes x megaman.png não batem).
- COVERS.BIN fragmentado (25/10/2026). Só as capas 0–3 apareciam: o arquivo no SD estava em 2 pedaços (clusters 0–9 e 10–13; clusters de 16 KB = 32 setores, e o cluster 10 começa na capa 4). A firmware (`lba_read`) faz base + setor e NÃO segue a cadeia do FAT. Com o arquivo fragmentado, a verificação [312..314] falha e a capa não aparece (falha segura). Causa: o COVERS.BIN foi gravado por cima do antigo. Solução: `ferramentas/copiar_sd.py` grava COVERSn.TMP reservando o tamanho todo (`truncate`), confere com `fsutil file queryextents` (funciona em FAT32), tenta até 4 vezes deixando os buracos ocupados, e só então troca o arquivo e avisa se ainda ficou fragmentado. Na ferramenta: "Cartão SD: [F:] Copiar para o cartão SD" (unidades com \EDFC, lembrada nas configurações). No SD do usuário: 1 pedaço (14 clusters). Correção definitiva possível: a firmware seguir a cadeia de clusters (`fat_next_cluster`), ainda não feita.
- Cartão SD por caminho (25/10/2026, pedido do usuário). No lugar da busca automática: "Cartão SD: [caminho] Adicionar caminho… / Remover / Copiar agora". Aceita a raiz (F:\) ou a pasta EDFC e fica lembrado em `gerador_config.json`. Com o caminho preenchido, ao gerar a firmware ela já vai para o cartão: o COVERS.BIN antigo é apagado antes (`gravar_continuo`) e o novo é gravado contínuo e conferido. A mensagem final inclui "CARTÃO SD: …" e vira aviso se ficar fragmentado. O COVERS.BIN da pasta de saída também é apagado antes de gerar. O caminho é lido na thread principal e passado para `rodar` (o Tk não pode ser lido de outra thread).
- OTIMIZAÇÃO COM NUMPY (25/10/2026, pedido do usuário). Backup antes: `BACKUP_ferramenta_antes_numpy_20261025/` (+ .zip). NumPy 2.5.4 (`pip install --user numpy`, Python 3.15). Novo `ferramentas/rapido.py`:
  - `tabela_dist` (dtable), `Unidades.erros` (uerr de todas as unidades do `optimize` de uma vez), `total_min`/`total_fixo`, `CustoTiles` (custo da fusão de tiles com memória: o mesmo grupo nunca é recalculado; antes todos os pares eram refeitos a cada fusão) e `optimize_frame` vetorizado em `sim_flicker` (tabela E pixel × cor, erro dos 48 blocos por paleta de uma vez).
  - Resultado IDÊNTICO bit a bit. Somas na ordem do Python com `np.cumsum(...)[-1]` (o `np.sum` soma em pares e muda o arredondamento). `x**2` e `x**0.5` feitos pelo próprio Python (`rapido.quad`): no Windows o `pow` do Python difere em ~0,2% dos casos, no último bit, do `x*x`/`sqrt` do NumPy. Empates resolvidos pelo primeiro.

- Conferência (`ferramentas/referencia_conversao.py`, código antigo x novo): 5 capas × 4 níveis, todas as partes e o registro do COVERS → 0 diferenças. As POCs λ 0,05/0,15/0,20/0,22 regeradas saem idênticas (`c79390f0`/`5c9e07cc`/`1a827fe8`/`dee4037b`). `teste_paletas_editadas` e `teste_capas_v3_emu`: TUDO OK.
- Tempos: conversão base (v2) 22 s → 0,6 s; um nível com flicker ~50 s → ~2,5 s; os 20 casos da referência 252 s → 15 s; o COVERS da POC (192 capas) ~5 s. Pesos da barra de % recalibrados (`PESOS_FLK`, `PESOS_CONV`).
- O cache dos níveis (`cache_flicker/`) continua válido, porque os resultados são os mesmos.

18. TESTE 80x96 (25/10/2026) — `ferramentas_80x96/`, `Gerador 80x96 (TESTE).bat`, `experimento_80x96/`

Backup da versão funcional antes: `FUNCIONAL_ferramenta_64x96_20261025/` (+ .zip). A ferramenta normal (`ferramentas/`) não foi tocada.

- Geometria: capa nas colunas 21-30 (x 168-247). Sprites como antes, nas colunas 22-29 (x 176). As colunas 21 e 30 só têm a camada de fundo. Blocos de atributo: começam na coluna 20 (vazia), então há um meio-bloco de 8 px em cada borda: 6 blocos por linha × 6 linhas = 36 (`bgb(x,y) = (y//16)*6 + (x+8)//16`). Fundo de 10 × 12 tiles (120 posições), ainda com o limite de 63 tiles únicos. `build_poc`: `COVER_W=80`, `SPR_X=8`, `bgb`/`spb`/`tem_spr` (a POC da capa fixa, base da firmware, continua com 64).
- Firmware (`build_capas_v3.py` da cópia, OS.BIN `5a23c7e5…`): `NT_BUF=$6095`. A carga copia 10 colunas por linha (contador em `cnt`, +22 por linha, NT de 320 a 439). `bg_clear` usa `LDY #9`, `upload_bg` usa `CPY #10`. Nomes com 18 colunas (`$CB29=$12`: a coluna 20 fica vazia e a 21 é da capa). Os atributos são os mesmos 19 bytes `$63CD-$63DF` (já cobriam as colunas 20-31). Metadados do COVERS: NT[120] em 320 e atributos[19] em 440. Espaço: SEG_MAIN 599/612.
- Conversor: `optimize_frame` soma também o erro das laterais (só dependem do fundo) no total. Na ferramenta, o conta-gotas nas laterais só oferece o fundo; não dá para pintar ponto lá (não há sprite).
- Separado da ferramenta normal: config `imagens/capas_config_80x96.cfg` (cópia inicial da normal; `.cfg` para a normal não listar como projeto), cores editadas em `imagens/paletas_editadas_80x96/`, cache próprio e saída `firmware_gerada_80x96/`. Projetos .json do Palette Lab (64x96) não entram. Sempre usa o OS.BIN de teste (capas "sem flicker" = A=B).
- Testes: `teste_80x96_emu.py` (NT/atributos no buffer e na PPU, texto 2-19 preservado, coluna 20 vazia, 31 intacta; imagem da PPU = prévia 7680/7680 px; limpeza no hide) e `teste_gui_80x96.py` (janela, conta-gotas lateral, geração = CLI): TUDO OK. Firmware pronta: `firmware_gerada_80x96/` (11 capas de `F:\roms`, COVERS `1ef5f6f8…`). **Hardware (25/10/2026): "ficou horrível, quebrou as capas".** Causa ainda não diagnosticada (falta foto). O SD estava com o OS `5a23c7e5` e o COVERS contínuo.

19. Investigação: bancos de CHR com o menu rodando (25/10/2026)

- No carregamento do jogo (v1.26 `$B471–$B4B8`, com o menu ainda rodando), para cada 8 KB de CHR o OS faz 2×: `$4400=$82`, `$4401=banco` (+1 a cada vez), `file_read_sectors(dest=$0FF0, 8 setores)`. Destino < `$2000` → `disk_read_to_ppu` (BIOS). Ou seja, bancos de **4 KB** escritos pela PPU. Em `$B512–$B5A0` o OS lê 16 bytes da PPU `$0FF0` no banco 0 e os regrava no banco N: o `$82` muda a memória que a PPU enxerga, com o menu rodando [INF]. No reset, `$82=0`.
- Ideia: trocar de banco no meio do quadro (por linha de tiles), para a região da capa ter tiles próprios. Isso acabaria com o limite de 63 tiles. NÃO resolve o limite de cores (8 paletas da PPU) nem o de 8 sprites por linha. Exige temporização por linha (NMI + espera ou sprite 0) e a fonte copiada em cada banco.
- Diagnóstico para o console: `ferramentas/build_diag_bancos.py` → `teste_bancos_chr/EDFC/OS.BIN` (`765d0ca0…`). É a v3 normal + `INC tick` no NMI + `diag` na espera ociosa (`i_done: JMP diag`), que alterna `$82` entre 0 e 1 a cada 64 quadros, fora do NMI (para não atrapalhar escritas do OS em `$4400`/`$4401`). Simulado: a capa carrega igual e as trocas acontecem nos quadros 63/127/191/255, só `($82, 0/1)`. **Hardware (25/10/2026, vídeo em `debug/video.mp4`): a tela inteira alterna** (texto e capa); o jogo abre normal. Com o banco 1: texto = blocos amarelos (os tiles sólidos da tabela 1 do OS.BIN) e a capa certa (ela foi carregada com o banco 1 ativo). Com o banco 0: texto normal e a capa embaralhada. Conclusão: o `$82` escolhe a janela de 8 KB da PPU a partir de `valor × 4 KB` (banco 1 = blocos 1 e 2), e cada banco é memória separada.
- POC da troca no meio da tela: `ferramentas/build_poc_bancos.py` → `poc_bancos/EDFC/OS.BIN` (`55199bdf…`, base v2 sem flicker, COVERS v2 das 11 capas de `F:\roms`).
  - Os tiles da capa só existem no banco 2 (`lc2` em volta do `load_cover`); a fonte fica nos bancos 0 e 2 (`chr2` chama o `$826F` duas vezes, patches em `$ECDB`/`$ED7B`).
  - Sincronia: sprite 0 em OAM[0] (Y 15, tile `$61` = tiles `$60`/`$61` da tabela 1, pixel (0,0) opaco, atrás do fundo, X 0). O sprite 0 da capa vai para OAM[48]. Toda a fonte é opaca (o "fundo" das letras é a cor 2 = preto), então o hit acontece em qualquer lugar e o sprite fica invisível.

- `split` na espera ociosa: espera `$9E` (NMI), a flag V apagar (fim do vblank) e acender (linha 16). Depois `dly` (Y×(5A+6)+15): 1753 ciclos → `$82=2` no ponto 190-223 da linha 31 (antes da busca dos sprites da linha 32, pontos 257-320) e 10923+12 → `$82=0` no ponto 259-292 da linha 127. Cada espera tem limite.
- Teste `teste_poc_bancos_emu.py`: fonte nos 2 bancos, tiles da capa só no banco 2, OAM, a ordem das trocas e os ciclos contados no código montado (1768 / 10935), e a desistência sem sprite 0. TUDO OK. **Hardware (25/10/2026): funcionou, a capa aparece perfeita.** Mas não há ganho visual: liberar tiles quase não muda a capa 64x96 (`debug/comparacao_tiles_bancos.png`: ΔE cai ≤ 1, 0,2–3,5% dos pixels). Simulações depois: núcleo de FPGA hipotético com cor por 8x8 + tiles livres (`sim_nucleo_fpga.py`, `debug/comparacao_nucleo_fpga.png`) e pontilhado (`sim_dither.py`, `debug/comparacao_pontilhado.png`). Ganho pequeno ou piora; o flicker atual rende mais. O limite está nas paletas da PPU.

20. POC: texto fixo no navegador (26/10/2026)

- `ferramentas/build_poc_texto.py` → `poc_texto/EDFC/OS.BIN` (`7d55caf6…`): é a v3 normal com "Instagram: @tavinho.games" na linha 23 da tela (col 3), logo acima da barra cinza. No navegador o OS usa as linhas 2-22 (lista, y 0-20) e 25-26 (nome completo, y 23-24); as linhas 0-1 e 23-24 ficam livres (y 21-22 só a tela de cheats usa, e ela redesenha a tela inteira sem passar pelo `w82fa`).
- 1ª versão (`9b5e6d39…`, linha 1 = pontos 8-15): no hardware o texto ficou escondido pela borda da TV de tubo (overscan). Por isso foi para a linha 23 (pontos 184-191), bem dentro da área visível.
- O texto é copiado para o buffer (`$62E3`) no `w82fa` (o wrapper do redesenho completo `$82FA`, `$CB83`), logo antes de o OS enviar a tela inteira. Tile = código ASCII (o OS faz `char + $FF + C`). 12 bytes de código + 25 do texto: SEG_E `$A680` ficou com 125/135.
- Teste `teste_poc_texto.py`: o texto vai para a linha 23 da PPU, o resto da tela fica igual ao buffer, e a capa carrega. Imagem de exemplo reconstruída da PPU: `debug/poc_texto_exemplo.png`. Hardware da linha 23: *pendente*.
- Tela 3: GERAR. Copia `ferramentas/firmwares/OS_v3.BIN` (`5b23f07e…`), ou `OS_v2.BIN` (`c0a74aea…`) no nível "v2", conferindo o SHA-1. Roda `gerar_covers.py --v3 --lambda X --jobs N` (ou `--v2`) e escreve `LEIA-ME.txt` com o nível e os hashes.
- `gerar_covers.py --jobs N`: conversão em paralelo (`Pool.imap`, ordem preservada; mesmo resultado byte a byte).
- `teste_gerador_gui.py <saida>`: abre a janela, mexe na barra e gera λ=0,22 (COVERS idêntico ao da POC, `dee4037b…`) e v2. TUDO OK.

21. Associação inteligente ROM ↔ capa (26/10/2026)

- A firmware acha a capa pelo CRC do nome do arquivo da ROM, então o registro precisa ser gravado com o nome EXATO da ROM. `ferramentas/associar_roms.py` escolhe, para cada ROM, a capa de título mais parecido: tira número de lista ("001 "), (…)/[…], "The", acentos e pontuação; "&"="and"; II..VIII=2..8; os números do título têm de bater (um "1" sobrando é tolerado); título contido no outro ou sem o subtítulo (" - …") também vale; empate: mesma região da ROM, depois EUA > Mundo > Europa > Japão, sem Beta/Proto/Pirata. Limiar 0,84.
- Teste: `001-200` + `super mario` contra os 2878 `Named_Boxarts`: 180 de 197 ROMs com capa, em 0,1 s. Os erros conhecidos: hacks recebem a capa do jogo original ("Castlevania - Skel's Revenge" → Castlevania) e "Batman 3 - Batman Returns" → "Batman III" (pirata). `python associar_roms.py <capas> <roms>` lista tudo.
- `gerar_covers.py --roms`: usa a associação (subpastas incluídas). Uma capa usada por várias ROMs é convertida uma vez só; as outras cópias só trocam os 3 bytes de verificação do CRC (setor 0: v1 272, v2 284, v3 312; conferido que são os únicos bytes que dependem do CRC). Avisa as ROMs sem capa e as imagens sem ROM.
- Ferramenta: botão "Mostrar só capas das minhas ROMs" na lista de Capas (busca em segundo plano). Mostra o resumo, a(s) ROM(s) da capa escolhida (≈ % quando o nome não é igual), "N ROMs" no item e a lista das ROMs sem capa. O modo é lembrado (`so_rom` nas prefs). Backup antes: `BACKUP_ferramenta_antes_assoc_roms_20261026/`.
- Nitidez "Definir padrão": grava `nitidez_padrao` (bool) e `nitidez_forca_padrao` (%) no `capas_config.json`; vale para as capas sem escolha própria (na ferramenta e no `gerar_covers.py`). A capa atual passa a seguir o padrão; as capas que seguiam o padrão e têm cores editadas ficam congeladas como estavam (a edição é por nitidez/força). Força igual à padrão = segue o padrão.

22. POC: capa em tela cheia (26/10/2026)

- Simulações antes: `sim_tela_cheia.py [força]` → `debug/comparacao_tela_cheia*.png` (64x96 de hoje × 128x192 × 176x240 × 176x240 com paletas por faixa). Ganho grande em tamanho/nitidez; cor continua limitada (4 paletas/16x16).
- `ferramentas/build_poc_tela_cheia.py` → `poc_tela_cheia/EDFC/` (OS.BIN `f2f1fef6…`, COVERS.BIN `18bce8c5…` = o da firmware_gerada + visualizador + 11 capas grandes para 18 ROMs). Conversão: `conv_tela_cheia.py` (176x240 em x=48, só fundo, fundo preto, nitidez 120%; 27 setores por capa:

"TELA"+paletas+sprite 0+esperas, NT+atributos, blocos de CHR 2/3/4).

- OS: `idle_hook` zera `fs_t` ao carregar a capa e vai para `fs_hook` (SEG_E): com `has`, conta quadros pelo `$9E` (`LSR`); no 180º lê o setor `FULL_SEC` (fim do COVERS original) e faz `JSR → JMP ($04E9)`.
- Visualizador (505/512 bytes, roda em `$6400` = backup da tela do OS, livre no navegador; variáveis em `$67E0`): copia-se do buffer do setor, procura `cur` no índice (setor seguinte: n + n×[CRC 4, setor 2]), confere "TELA", `flk=0`, tela e NMI desligados, envia NT (2 setores), blocos 2+3 (banco 2, `$0000`, 16 setores) e bloco 4 (banco 3, `$1000`, 8 setores), paletas (mesmas 16 no fundo e nos sprites), OAM só com o sprite 0. Quadro: espera `$9E`, botão → sai; banco 2; V apaga/acende (sprite 0 no 1º pixel não preto do topo, atrás do fundo, tile `$FF` = `$FE/$FF` do bloco 3); `v_dly` → banco 3 (~linha 75), `v_dly` → banco 4 (~linha 155). As linhas de tiles 9 e 19 estão nos dois blocos vizinhos com os mesmos números: a troca vale em qualquer ponto das linhas de pixels 71-79 / 151-159 (janela de ~8 linhas, não de 64 pontos). Saída: espera soltar, banco 0, paleta `$825F`, `$82FA` (NT do buffer), `$2001=$0A`, `JMP mask_entry`.
- Teste `teste_poc_tela_cheia.py`: contador (179 não, 180 sim), jogo sem tela cheia volta sem mexer, CHR nos blocos certos, NT/paletas/OAM, ciclos contados → banco 3 na linha 75 ponto 282, banco 4 na 155 ponto 311, tela reconstruída = imagem convertida com a troca em qualquer linha das janelas, saída. TUDO OK. Imagem: `debug/poc_tela_cheia_exemplo.png`. Hardware: *pendente*. Riscos: blocos 4/5 da CHR (banco 4) nunca testados; `$6400` como memória de trabalho.

15. Refazer a análise

```
"C:\tools\ghidra_12.1.4_PUBLIC\support\analyzeHeadless.bat" ^
  "<pasta do projeto>\analise\ghidra_proj" EDFC_OS ^
  -import "<pasta do projeto>\analise\prg_8000.bin" -overwrite ^
  -processor 6502:LE:16:default -loader BinaryLoader -loader-baseAddr 0x8000 ^
  -scriptPath "<pasta do projeto>\ferramentas\ghidra_scripts" ^
  -postScript DumpNES.java "<pasta do projeto>\analise\ghidra_out" ^
  "<pasta do projeto>\ferramentas\ghidra_scripts\nomes_n8.txt"
```

23. POC v4: capa grande no navegador + tela cheia (26-27/10/2026) — FUNCIONOU no aparelho

- `build_capas_v4.py` + `build_poc_capa_grande.py` → `poc_capa_grande/EDFC/` (821 capas, 25,9 MB). Capa 112x160 só de fundo nas linhas 2-21, colunas 16-31 (nomes com 14 colunas, `$CB29=$0E`); paletas 0 e 3 fixas, 1 e 2 da capa. Tiles nos slots livres da fonte de 7 blocos (bancos 1-7), um grupo de linhas por banco, linhas compartilhadas (folga de 8 linhas na troca). `chr_all` = `$826F` com `$82` = 0..7. Split na espera ociosa: banco 1 no vblank, sprite 0 na linha 16 (coluna 0, atrás do texto), 6 trocas de 24 em 24 linhas. Aparecimento em 7 quadros (atributos pretos → 4 linhas de NT por quadro → atributos/paletas). Botão visto no vblank: esconde na hora. 180 quadros parado: tela cheia (visualizador no setor 256, dados em csec+15). `teste_capas_v4_emu.py`: TUDO OK.

24. POC v5: + sprites por cima da capa grande (27/10/2026)

- Simulação `debug/comparacao_sprites_capa_grande.png`: ΔE médio cai 15-23% (rosto do Mario, Drácula, Mega Man).
- `conv_sprites_grande.py`: até 62 sprites 8x16 (grade de 16 linhas x 8 pontos, máx. 8 por faixa, 7 na faixa do sprite 0), 4 paletas (12 cores), cada ponto transparente ou cor (só onde melhora). `conv_capa_grande.converter_v5`.
- `build_capas_v5.py`: fundo nos bancos ímpares 1..13 (bloco 2g+1); bloco 2g+2 = tiles de sprite daquele trecho (cada par vai para os blocos que podem estar ativos nas linhas dele: máscara; mesmo número em todos). Sprite 0 = par 0 do bloco 2. Fonte: `$826F` com 0,1,3..13. Módulo de 1 KB (setores 257-258) em `$6400` recarregado a cada capa (lmod residente); OAM direto em `$0200` com paletas de sprite pretas até o fim do aparecimento; variáveis em `$7F00`. Residente sobrando: 227 + 109 bytes. Registro: setor 42 = OAM + paletas + máscaras, 43-46 = 63 pares.
- `build_poc_capa_grande_v5.py` (5 capas de teste) → `poc_capa_grande_v5/EDFC/` (OS.BIN `79894e96…`). `teste_capas_v5_emu.py`: 37 verificações, TUDO OK (tiles nos blocos certos, 118 cópias de sprite, trocas nas linhas 42-162, tela reconstruída = convertida com a troca em qualquer ponto da folga). Hardware: *pendente* (risco: blocos de CHR até o 14).
- Tela cheia removida da v5 (pedido do usuário; não afeta a capa do navegador): sem visualizador, sem contador, registro de 32 setores (capa i em 512 + 32*i; sprites nos setores 15-19). Residente sobrando: 227 + 132 + 18 + 13 + 8. `teste_capas_v5_emu.py`: 33 verificações, TUDO OK (inclui 300 quadros parado sem ler o cartão).
- Hardware (27/10/2026): às vezes lixo ao trocar de capa (fotos em `imagens/novos testes`): letras trocadas e pontinhos nas linhas de cima/coluna 0, capa embaralhada. Causa: a carga dos tiles de sprite escrevia 3 pares por vblank e, com o laço das máscaras, passava do vblank (até 2203 ciclos depois do início; o limite é ~1850 porque o NMI do OS já gasta ~330) -> escritas na PPU durante o desenho caem em lugares aleatórios da nametable/CHR. Correção: 2 pares por vblank (pior caso 1623). `ciclos6502.py` + o teste agora medem os ciclos de TODA escrita na

PPU com a tela ligada (desde o WAITVB/NMI; leitura do cartão no meio exige vblank novo) e falham se passar.

25. POC v6: a v5 carregando mais rápido (27/10/2026) — a v5 fica como estável

- `build_capas_v6.py` (só o módulo muda) + `conv_capa_grande.v6_de_v5` + `build_poc_capa_grande_v6.py` → `poc_capa_grande_v6/EDFC/`. (1) `vbw`: espera o vblank pelo `$9E` do NMI do OS (o WAITVB lê `$2002`, que o NMI também lê e apaga: perdia vblanks); limite de ~2 quadros e, sem NMI, cai no WAITVB. (2) Sprites: setor 15 = OAM + paletas + tabela de 7 x (setor, pedaços, destino); os pares de cada bloco são uma faixa contínua, gravada em sequência a partir do setor 16 e enviada pelo mesmo `l_sec` do fundo. (3) Fundo: `meta[353:367]` = pedaços de 128 bytes de cada grupo (A/B) pelos slots que a nametable usa (Mega Man 2: 47 de 56).
- `teste_capas_v6_emu.py`: 35 verificações, TUDO OK; carga = 79 esperas pelo NMI e 0 pelo WAITVB (v5: ~116 WAITVB); pior escrita na PPU 1635 ciclos depois do início do vblank. Hardware: *pendente*.

26. POC v7: vblank estendido na carga (27/10/2026) — a v6 fica como estável

- `build_capas_v7.py` + `build_poc_capa_grande_v7.py` → `poc_capa_grande_v7/EDFC/`. Carga: `xvb` = espera o NMI (`vbw`) + `dly` até a linha ~225 da tela seguinte ((262-241+225) linhas - 330 ciclos já gastos no NMI) e `$2001=0`; manda 2 pedaços de 128 (`up128`); `xvb_fim` = `scroll0` + `$2001=$0A` antes do pré-render (linha 261, ponto 257; o NMI da linha 241 roda dentro da janela). Nametable + atributos ($55) + paletas 1-2 pretas também na carga (`nt_hid`, 3 quadros) -> o aparecimento começa em `rev=7` (1 quadro). Módulo com marca "M7OK" em $67FC: `lmod` não relê se ela está lá.
- `teste_capas_v7_emu.py` (39 OK): carga 43-47 quadros nas 5 capas (v6 ~85); tela desligada na linha 224,8 (sempre abaixo da barra, que acaba na 223) e religada com ~4 linhas de folga; escritas no vblank normal ≤1316 ciclos (inclui `$2000/$2005`); recarga não relê os setores 257-258. Hardware: *pendente*.
- Hardware da 1ª v7 (fotos `imagens/testev7`): lixo na linha 0, numa linha da lista e no topo da capa. Causa: a janela (linha 225 até o pré-render) atravessava a linha 241; o NMI do OS lê `$2002` e zerava a meia escrita do `$2006` no meio dos pares da nametable (`nt_rows`) -> linhas no endereço errado. E contar do NMI até a linha 225 da tela SEGUINTE dava 1 janela a cada 2 quadros (sem ganho real). O teste antigo simulava o NMI por contagem de leituras e não pegou nada disso.
- Correção: `xvb` espera o sprite 0 da CARGA (OAM só com ele: Y 199, tile `$00` = sólido em todos os bancos de fonte, atrás da célula `$00` da coluna 0 da barra) -> linha 200 -> `dly` até a 226 -> `$2001=0` e `$2000=$20` (NMI desligado na janela); `xvb_fim`: `BIT $2002`, `$2005` x2, `$2001=$1E`, `$2000=$A0` (sem NMI espúrio). Sem sprite 0: apaga no vblank e religa no próximo. `ld_oam` monta a página antes do `vbw` (o OAMDMA saía tarde).
- `ppu_tempo.py`: modelo de tempo para os testes (linhas, NMI de verdade do OS na 241 interrompendo o código, sprite 0 na linha do OAM[0], `$2002` apaga vblank e latch, WAITVB = salto até o vblank, joypad lido pelo NMI). `teste_capas_v7_emu.py` reescrito com ele: ~46 quadros por capa (v6 no mesmo teste: ~77 + 6 do aparecimento), nametable inteira = buffer nas 5 capas, janelas abaixo da barra, nenhuma escrita fora do vblank. TUDO OK.
- Hardware da 2ª v7: travava ao parar num jogo. Causa provável: `lmod` reaproveita o módulo se a marca no fim ($67FC) bater; as duas v7 usavam "M7OK" e a WRAM não é zerada no reset -> a nova rodava o módulo VELHO com os endereços novos. Agora a marca = 4 primeiros bytes do SHA-1 do módulo, gravada no COVERS.BIN e no `lm_mg`. Blindagens: a tela é desligada no intervalo horizontal (linha 225, ponto ~280; no meio da linha a PPU pode corromper a OAM) e cada janela refaz o OAMDMA da OAM da carga (o sprite 0 da linha 200 não se perde); por isso a nametable escondida vai antes dos sprites e a OAM de verdade só no fim do `spr_load`. Teste: módulo velho com "M7OK" na WRAM -> a 1ª carga relê o módulo; desliga no ponto 268-301 da linha 225. TUDO OK.

27. POC v8: flicker opcional nos sprites, por capa (27/10/2026) — a v7 fica como estável (funcionou)

- `conv_sprites_grande.otimizar_flicker`: parte da solução sem flicker (mesmas células/paleta por sprite), dá a cada sprite um quadro B (paletas B, tile B); cada ponto = média em luz (A, B), cada um = fundo ou cor da paleta; custo $\Delta E^2 + \lambda \cdot \Delta E^2(A,B)$, λ=0,05 (0,25 não ganhava nada). Onde há sprites o ΔE cai ~25-35% (`debug/comparacao_flicker_sprites.png`). `conv_capa_grande.converter_v8(..., flicker)`: registro de 64 setores; setor 15 = OAM A + [256] paletas A + [272] faixas A + [300] flag + [304] paletas B + [320] faixas B (pares p+64, tile = 2p+1 EOR $80).
- `build_capas_v8.py`: `spr_load` copia os bytes 256-347 para `$7F80` e carrega as faixas A e, com flag, as B; `flk_vb` (no vblank do split, só com a capa pronta) escreve as paletas do quadro e faz o OAMDMA; `flk_tg` (no fim do split, no módulo) troca os tiles da OAM ($80, menos o sprite 0) e o quadro. `setP`→`$FBD4`, `q_next`→`$F753`.
- `teste_capas_v8_emu.py` (tempo real): tiles e paletas alternam a cada quadro; os 2 quadros = os do conversor; a média = a imagem prevista (diferença 0); capa sem flicker não muda; carga 45-65 quadros; nada fora do vblank. Hardware: *pendente*.

28. POC v9: cor livre da paleta 3 (27/10/2026)

- Fonte do OS: texto normal usa a cor 1, destaque/pastas a cor 3; paleta 0 = lista (cor 3 = amarelo das pastas e do cursor), paleta 3 = barra (1 = branco do nome do jogo, 2 = cinza, 3 = amarelo do nome de pasta). Com um jogo selecionado (capa na tela) a cor 3 da paleta 3 fica livre.
- `conv_capa_grande.quantizar(img, cor3=True)` otimiza também essa cor; `converter_v9` converte com e sem e fica com a melhor (nunca piora); byte 367 do setor 0. Ganho: Mega Man 2 ΔE 19,8 -> 18,7 ($05); Castlevania, Contra e Zelda pequeno ($18); Mario 3, Metroid e Ninja Gaiden ficam com $28.
- `build_capas_v9.py`: `l_ct` copia 15 bytes (CHT + COR3 em $7F6E); `pal_wr` escreve $3F0F = COR3 (ou $28 ao esconder); `hide` ($833D) regrava $3F0C-$3F0F do OS. Espaço: `lm_mg`→$81C5, `q_next`→$D0C6, `frm_tg`→$A680, `sp_all` antes do `spr_load` (sem o JMP). `teste_capas_v9_emu.py` (tempo real): TUDO OK, inclusive $3F0F = cor da capa com ela na tela e = $28 depois do botão e do menu. Hardware: *pendente*.

29. POC v10: espera antes de carregar + cancelamento (27/10/2026) — a v9 funcionou no aparelho

- `build_capas_v10.py`: `idle_hook` chama `i_wait` (SEG_E): `pending` vira contador de quadros (o OS o põe em 1 a cada movimento do cursor); carrega quando `pending >= espq` (`espq` = quadros + 1, padrão 31; byte em $A706 = posição 34566 do OS.BIN, para a ferramenta trocar). Espaço: o split espera o vblank pelo `vbw` do módulo (-14 bytes) e os passos 1-6 do aparecimento (mortos desde a v7) saíram do módulo.
- Cancelamento: o NMI fica desligado nas janelas da carga e não lê o controle -> `joyp` lê `$4016` direto antes de cada envio (`l_fr`); botão -> `l_desiste`: `TXS` com a pilha guardada no começo de `lc_rest`, `$2001=$0A`, `has=0`, `rev=0`, `pending=1`, `bgshown=1`.
- `teste_capas_v10_emu.py` (tempo real; `ppu_tempo` ganhou `joy_em` = botão a partir de um instante): a carga começa ~29-30 quadros depois de parar; 20 quadros parado não carrega; botão no meio: desiste em 0,3 quadro com a pilha intacta, tela/NMI ligados e a lista intacta; parado de novo recarrega inteira. TUDO OK. Hardware: *pendente*.
- Hardware da 1ª v10: a caixa do File Menu ficava cinza só até a coluna 15 (foto `imagens/novos testes/4.jpeg`). Causa: desde a v4 o `bg_clear` zerava os atributos da região da capa no buffer; o `hide` ($833D) roda depois de o OS desenhar o menu e apagava a paleta 3 da caixa nas colunas 16-31. (Na v2 a limpeza preservava a paleta 3, mas agora a capa também usa a 3.) Correção: os atributos da capa não vão mais para o buffer do OS — ficam em `ATC` ($7F00, 6 x 4 de 8 em 8) e `at_ppu` os envia direto; `bg_clear` só limpa as células (o buffer de atributos é só do OS). Teste: caixa de menu com paleta 3 sobre a região da capa continua inteira; nametable conferida com os atributos da capa só na PPU. TUDO OK.
- Hardware da v10 corrigida: **funcionou** (27/10/2026).

30. A v10 na ferramenta (27/10/2026)

- `gerador_firmware_capas.py` ganhou o seletor **Formato** no topo da seção 2: *Capa grande (v10)* (novo padrão) ou *Capa pequena (v3)* (tudo como antes: barra de λ, cores editáveis, flicker por nível). O formato e a espera ficam em `gerador_config.json` (`formato`, `espera`). Backup da ferramenta de antes: `BACKUP_ferramenta_antes_v10_20261027/`.
- Capa grande: prévia 112x160 (original com a nitidez, capa no NES com a escolha desta capa animada A/B no vsync, capa sem flicker), nitidez + força por capa (as mesmas chaves `nitidez`/`nitidez_forca`; desligada = força 0), **flicker nos sprites por capa** (`flicker_grande` / `flicker_grande_padrao` no `capas_config.json`; "Definir como padrão" e "Usar o padrão nesta capa" valem para ele) e a **espera** (0 a 4 s, de 0,1 em 0,1). Projetos .json do Palette Lab não servem para a capa grande (usa-se o PNG de mesmo nome).
- Geração: `firmwares/OS_v10.BIN` (SHA 4c7281b9…, = POC v10) e `firmwares/MOD_v10.BIN` (módulo de 1 KB com a impressão digital, SHA 6e5e9cbf…), conferidos pelo SHA; a ferramenta grava `quadros + 1` no byte 34566 (`espq`, $A706) — só esse byte muda; a impressão digital do módulo não depende dele. COVERS.BIN: `gerar_covers_v10.py` (mesma associação de ROMs e mesmas linhas de progresso/AVISO do `gerar_covers.py`; as capas com flicker, ~1,5 min cada, convertem primeiro; sem flicker ~1 s). Limite: 1015 capas (setor de 16 bits: 512 + 64*idx + 63 < 65536).
- Testes: `gerar_covers_v10.py` com as capas/ROMs/opções da POC dá o COVERS.BIN idêntico ao da POC (0e495260…); `teste_gerador_gui_v10.py` (janela de verdade): prévias, flicker por capa salvo, troca de formato, geração com 0,5 s = OS.BIN e COVERS.BIN da POC v10, 1,2 s muda só o byte espq (73), preferências lembradas, sem exceções. TUDO OK. Teste de tempo real da v10 com o byte da espera trocado para 72 quadros (1,2 s): a carga começa em ~71 quadros, cancelamento e o resto iguais. TUDO OK.
- 28/10/2026: a capa pequena (v3) saiu da ferramenta a pedido do usuário (seletor de formato, barra de λ, editor de cores, projetos .json): ela só gera a v10. Barra de rolagem à direita no corpo da janela; zoom 1-3×. Backup com a v3:

`BACKUP_ferramenta_com_v3_20261028/` (inclui o teste_gerador_gui.py antigo). `gerar_covers.py` e `firmwares/OS_v2/OS_v3` continuam no disco (usados pelas POCs antigas).

- 28/10/2026: conta-gotas e barra de nível de flicker na capa grande. `capa_grande_cores.py`: a conversão devolve a estrutura (info['est']: índices do fundo, paleta de cada bloco, sprites A/B, paletas); recolorir = só trocar paletas (setor 0 [3:9] e [367], setor 15 [256:272] A e [304:320] B), a prévia recolorida é idêntica ao conversor (diferença 0). Edições em <capas>/paletas_editadas_grande/<capa>.F<força>.<sem|L<λ>>.json = {orig, novo} (só valem com as mesmas paletas originais). Fixas: paleta 0, branco/cinza da barra, preto. Níveis do flicker (λ): 0, 0.02, 0.05 (padrão), 0.10, 0.15, 0.20, 0.25 (medido: em 0.40 o flicker some; tremido médio Castlevania 23 -> 0, ΔE 13.7 -> 16.4). Config: flicker_grande_lambda / flicker_grande_lambda_padrao. A ferramenta calcula os 7 níveis da capa em segundo plano (o escolhido primeiro) e a prévia sem flicker num processo só dela. Teste: teste_gerador_gui_v10.py (sem edição = POC v10 idêntica; edição muda só os bytes de paleta; λ muda só o registro da capa). TUDO OK.
- 28/10/2026: pontilhado (dithering) opcional por capa (`conv_capa_grande.pontilhar`): xadrez Bayer 2x2, cada ponto = mistura (na luz) de 2 das 4 cores da paleta do bloco em 0/25/50/75/100%, custo ΔE² + PEN_PONT(0.05) * ΔE²(cor1, cor2) * 4t(1-t). Os sprites otimizam contra o fundo percebido (a mistura). Não gasta tiles a mais (máx. 57 de 63 por grupo). Medido com borrão de TV composta (`capa_grande_cores.tv`): ΔE na TV Zelda 12.7->10.4, Mega Man 2 12.6->11.3, Castlevania 10.3->9.8, Contra 8.5->8.1, Mario 3 16.0->15.7 (`comparacao_pontilhado.png`). Config `pontilhado`/`pontilhado_padrao`; edição de cores com sufixo .P. Ferramenta: checkbox 'Pontilhado (nesta capa)' + 'como na TV (borrão do composto)' na prévia. Tempo real com as 5 capas pontilhadas: TUDO OK (média 54.6 quadros, nenhuma escrita fora do vblank, tela = pontos do conversor).
- 28/10/2026: 'Só acrescentar capas novas' (ferramenta, seção 3) / `gerar_covers_v10.py --base COVERS.BIN`: lê o índice do COVERS.BIN anterior (crc = balde | 3 bytes << 8), copia os registros dele byte a byte no mesmo lugar, converte só as ROMs cujo CRC não está no índice, acrescenta no fim e refaz o índice. Recusa se o módulo (setores 257-258) não for o desta firmware. A ferramenta guarda a cópia de antes em <saída>/COVERS_anterior.BIN. Capas já existentes não são atualizadas (para mudar uma, gerar completo). Teste: TUDO OK.
- 29/10/2026: flicker ~8x mais rápido (`conv_sprites_grande.otimizar_flicker`): a busca de cada cor testa as 54 candidatas de uma vez (busca()), calcula uma vez só o que não muda, e as misturas candidata x paleta do outro quadro (iguais em todos os pontos) uma vez por busca. Castlevania 90-118 s -> 11,5 s sozinho. Resultado idêntico byte a byte (5 capas x λ 0.05/0.15) e a geração completa = POC v10. Teste completo da ferramenta: ~20 min -> 3 min. Backup: BACKUP_conversor_antes_flicker_rapido_20261029/. (Testado e descartado: mais tentativas nas paletas do fundo e fundo+sprites juntos: ganho < 1,5%, às vezes pior.)

31. POC v11: "@tavinho.games" embaixo da capa (29/10/2026) — a v10 fica como estável

- `build_capas_v11.py` = v10 + wrapper `w82fa` no redesenho completo (`$CB83: JSR $82FA` -> `JSR w82fa`, `$F879`) que copia "@tavinho.games" para a linha 23, colunas 17-30 do buffer (`$62F1`: centralizado embaixo da imagem da capa) antes de o OS enviar a tela. 12 bytes de código + 14 de texto no SEG_C (322/323).
- Atributos: a linha de atributos 5 (linhas 20-23, colunas 16-31) é da capa. Com a capa à vista os atributos dela já põem 22-23 na paleta 0 (`atributos()`: bloco 11 = 0); para esconder, `at_ppu` escrevia `$55` (paleta 1, preta) na linha toda e o texto sumia. Agora, com X >= 40 (linha 5), escreve `$05` (20-21 paleta 1, 22-23 paleta 0). +6 bytes no módulo (989/1020): impressão digital nova (módulo SHA `0225e7ef…`). O COVERS.BIN da v11 difere do da v10 só nos setores 257-258 (conferido).
- 1ª tentativa (à esquerda, linha 23 colunas 1-14, OS `e298ea2e…`, módulo da v10 inalterado): o usuário preferiu embaixo da capa.
- Remover o item "About" do menu foi avaliado: ~8 bytes espalhados (o texto + o ponteiro na tabela `$FB39`, que é copiada para a RAM; o menu é código genérico do C): não compensa.
- `poc_capa_grande_v11/EDFC`: OS.BIN `dfede28f…` + COVERS.BIN 5 capas `2cb13e4c…`; `completo_817_capas/EDFC`: o mesmo OS.BIN (espera 0,5 s, a do usuário) + o COVERS.BIN de `firmware_gerada` (817 capas, v10) com o módulo trocado (`884336f4…`). Testes: `teste_capas_v11_emu.py` (tempo real; esconder = `$55`x20 + `$05`x4): TUDO OK; wrapper no emulador: a linha 23 sai com o texto nas colunas 17-30 e nada mais muda. Hardware: *pendente*.

32. POC v12: 20 jogos por página (29/10/2026) — a v11 fica como estável

- `build_capas_v12.py` = v11 + o tamanho da página do navegador 21 -> 20, trocado no lugar (13 operandos, nenhum código novo): `browser_draw_list` $C9A3/$C9AB (início da página = sel/21*21) e $C9DB (laço das linhas); navegação $DA0F/$DA17 (início da página), $DA48 (↑ na 1ª linha -> última: 20 -> 19), $DA9E (↓ depois da última), $DAE5 (← → só com mais de uma página: 22 -> 21), $DAFC (→ +21), $DB55/$DB62/$DB6A (← da 1ª página -> última), $DBA6 (← -21). Não são da página (ficam): $D9CD/$D9FB (atraso da repetição do direcional, 20 quadros), as linhas da tela de cheats ($858D/$85A1), $920B (FAT), $B954, $CCBA, $CF76.

- Quirks do OS original (iguais na v12): com 256+ itens, ← → não trocam de página (o `BCS` de $DAE7 usa o carry que sobrou do `CMP #4` de $DA8D, que é 0 para ← e →) e a última página sai cheia (o fim da lista compara só o byte baixo da posição).
- `teste_pagina_v12.py`: roda o trecho de navegação $DA06-$DBB0 do próprio OS no emulador (com o runtime do cc65) para listas de 1 a 817 itens, toda posição, as 4 direções: v11 = modelo com página 21 e v12 = modelo com 20 (10204 casos cada); e `browser_draw_list` desenhando 21 (v11) / 20 (v12) itens com a pilha do C intacta. TUDO OK. Tempo real (`teste_capas_v11_emu` com a v12): TUDO OK. O módulo não muda: COVERS.BIN = os da v11.
- `poc_capa_grande_v12/EDFC` (OS `768f9cc1…` + 5 capas) e `completo_817_capas/EDFC` (OS com a espera do usuário + as 817 capas da v11). Hardware: **funcionou** (29/10/2026), 20 por página e o texto embaixo da capa.
- 1º teste no aparelho: travou ao abrir qualquer pasta a partir de F:\. Isolamento: v11 sozinha (texto) e v10 + só os 20 por página (`poc_teste_isolamento/`) não travaram; o OS.BIN da v12 conferido = v11 + os 13 bytes. Causa: a cópia do COVERS.BIN (26 MB) ficou FRAGMENTADA no cartão — o EverDrive lê o arquivo pelos setores contando que ele é contínuo, então o módulo (setores 257-258) veio de outro lugar e a firmware rodou lixo na 1ª carga (ao entrar numa pasta com jogos). Copiando com `copiar_sd.py` (apaga o antigo, grava contínuo e confere): não travou mais. **Sempre copiar com o copiar_sd.py / botão "Copiar agora".**
- Visto na análise (OS original): a leitura da pasta para em ~255 itens (`fat_dir_scan`: `0xfd < count`) — pastas com mais arquivos só mostram os primeiros ~255.

33. POC v12b: sem o "lixo" ao ligar (29/10/2026)

- Ao ligar, ~0,5 s de "lixo" (a tela toda com o mesmo padrão, piscando) antes do texto de carga (vídeo `debug/teste2.mp4`); o OS original não tem.
- 1ª tentativa (não resolveu): a 2ª tabela de tiles do OS.BIN (arquivo `$1000-$1FFF`) guardava a capa de teste da POC v1 (`build_poc.py`: tiles `$1000-$15FF`, paleta `$1EF0`, OAM `$1F00`), herdada por todas as versões e sem uso desde a v4. `build_capas_v12.chr_original()` a devolve ao original (fica como limpeza).
- Causa: o `chr_all` (fonte nos 8 bancos: 0, 1, 3, ..., 13) chama o `$826F` do OS, que espera o vblank, desliga a tela, prepara o banco e RELIGA a tela no fim (`$82F0: LDA #$0A / STA $2001`). Entre as chamadas o `chr_all` trocava de banco com a tela ligada, e o próximo banco (ainda com o conteúdo aleatório da CHR-RAM ao ligar) aparecia até o próximo `$826F` desligar a tela: ~1 quadro de lixo por banco (8 trocas). O OS original prepara 1 banco só.
- Correção: bancos em ordem inversa (13, 11, ..., 1 e o 0 por último), `$2001 = 0` antes de cada troca; só o último `$826F` (banco 0, o que fica na tela) deixa a tela ligada. O código novo é 4 bytes menor: completado com `.byte 0,0,0,0` para nada mudar de lugar (o módulo continua o da v11, `0225e7ef…`, e os COVERS.BIN valem).
- `teste_ligar_v12b.py`: roda o `chr_all` com o `$826F` real; acusa troca/ligar com a tela mostrando banco não preparado: v12 antiga = 8 trocas com lixo, v12b = nenhuma. Tempo real (`teste_capas_v11_emu`, que agora confere o conjunto dos bancos, não a ordem): TUDO OK.
- `poc_capa_grande_v12b/completo_817_capas/EDFC`: OS `6c27725b…` + COVERS.BIN da v11/v12. Hardware: **as capas bugaram** (30/10/2026); o usuário preferiu abandonar a correção (o lixo ao ligar não importa). **ABANDONADA**: o build_capas_v12.py voltou à v12 (OS 768f9cc1…, conferido), o teste virou teste_ligar_v12b_ABANDONADO.py. Suspeita não verificada: a tabela \ original (\) — os tiles de sprite que as faixas da capa não sobrescrevem passariam de vazios a blocos sólidos.

34. v12 trava ao abrir pastas; POC v13 = v10 + texto + 20 por página (30/10/2026)

- A v12 (v10 + T texto + M módulo com $05 + P 20 por página) travou ao abrir QUALQUER pasta, toda vez, com os arquivos certos e o COVERS.BIN contínuo no cartão (conferidos daqui: SHA e `copiar_sd.trechos` = 1). A conclusão da seção 32 (fragmentação) estava errada ou incompleta.
- Isolamento no aparelho: T+M (v11) ok; P (v10 + 20) ok; C1 = T+P ok (e o texto NÃO some com a capa escondida: a mudança M era desnecessária — a análise da seção 31 estava errada); C2 = M+P ok. Só T+M+P trava; causa não achada (no emulador, `teste_abrir_pasta.py`, a v12 abre pastas normalmente).
- `build_capas_v13.py` = C1: OS `96d89601…`, módulo = o da v10 (`6e5e9cbf…`): os COVERS.BIN da v10 valem. Tempo real (`teste_capas_v10_emu` com a v13): TUDO OK. Hardware (como C1): **funcionou**.
- `build_teste_isolamento2.py` monta C1/C2. A v11/v12 (módulo `0225e7ef…`) ficam só como registro.

35. A v13 na ferramenta (30/10/2026)

- Backup da versão estável (ferramenta + firmware v10 + o COVERS.BIN de 817 capas): `BACKUP_ESTAVEL_v10_ferramenta_20261030/` (sem os caches). Para voltar: copiar a pasta `ferramentas` do backup por cima.
- `gerador_firmware_capas.py`: a firmware de referência passou a ser `firmwares/OS_v13.BIN` (`96d89601…`); a espera continua no byte 34566 (quadros + 1). O módulo (`MOD_v10.BIN`), o `gerar_covers_v10.py` e o formato do

COVERS.BIN não mudam: os COVERS.BIN gerados antes valem, e o "Só acrescentar" aceita os da v10. Textos do LEIA-ME: "v13".

- `teste_gerador_gui_v10.py` (agora espera o OS da v13): TUDO OK.

36. A ferramenta em .exe (30/10/2026)

- PyInstaller 6.22 (Python 3.15), pasta única (--onedir, --windowed): compilado/dist/NES N8 Covers Editor/ (o .exe + _internal com firmwares/OS_v13.BIN e MOD_v10.BIN), 72 MB; compilado/NES N8 Covers Editor.zip (32 MB) com o LEIA-ME. Recompilar: ferramentas/compilar_exe.py.
- Mudanças para o .exe: CONGELADO (sys.frozen): ROOT = pasta do .exe, configurações em %APPDATA%/NES N8 Covers editor; o gerador de capas roda num multiprocessing.Process do próprio programa (linhas_gerador/_gerador_processo, com a saída linha a linha por uma fila) em vez de python gerar_covers_v10.py; freeze_support(); conv_capa_grande.fonte0 lê o tile 0 do OS_v13.BIN se não houver firmware_base. Modos escondidos para testar o .exe: --gerar-covers e --testar-geracao.
- Testes: a ferramenta (Python) com GERADOR_EM_PROCESSO=1: TUDO OK; o .exe gerou o COVERS.BIN da POC idêntico (0e495260...) pelos dois caminhos; a janela do .exe abre com as pastas do usuário e salva as configurações.

37. POC v14: ← → levam o cursor para o topo da página (30/10/2026)

- O OS faz sel = início da página nova + linha (a linha é guardada em c_sp[2..3] em $DA32). `build_capas_v14.py` = v13 + a linha trocada por 0, no lugar: $DB17 (→) e $DB73 (← da 1ª página para a última): `LDY #3 / JSR $F145` -> `LDA #0 / TAX / NOP / NOP`; $DBA2 (← nas outras, sel - 20): `LDA $8259 / SEC` -> `LDY #4 / LDA ($28),Y` (o início da página - 20; o carry já vem 1 do `BCS` de $DB57; o byte alto não muda, a pasta tem no máximo 254 itens).
- Detalhe herdado do OS: ← na 1ª página vai para n/20*20; com n múltiplo de 20 essa "página" fica vazia e o cursor vai para o último jogo (o fim da última página de verdade).
- `teste_pagina_v14.py` (a navegação do OS no emulador, listas de 1 a 254, toda posição, 4 direções, 12232 casos): TUDO OK. Tempo real (`teste_capas_v10_emu` com a v14): TUDO OK. OS `876d21bf…`, módulo da v10. Hardware: *pendente*.

38. POC v15: o travamento ao carregar capas depois de ligar (30/10/2026)

- Sintoma (cartão recém-formatado, v13 e também a v10 estável): ao ligar, abrir uma pasta e parar num jogo trava; RESET e abrir um jogo depressa (antes da capa) "conserta"; o boot seguinte funciona, o próximo trava de novo.
- Causa: o módulo ($6400-$67FF) fica na WRAM com BATERIA e sobrevive ao desligar. Em toda inicialização o OS (os_init $BBB1, sem condição: `flash_cmd_stub_save_ram` $84C0) usa $6400-$66FF de rascunho (copia $0400/$0500 para lá, roda um stub em $0400 e devolve) e estraga os primeiros 768 bytes do módulo, sem tocar em $6700-$67FF, onde fica a marca ($67FC) que o lmod conferia. Marca intacta + corpo estragado => o lmod não relia e a 1ª carga pulava para o código estragado. Abrir um jogo apaga a WRAM inteira (a marca também) => o boot seguinte relia (o "alterna"). Bug latente desde a v5 (módulo em $6400); aparecia só quando o módulo sobrevivia de uma sessão para a outra sem jogo no meio. (Provavelmente também era o "trava ao abrir pastas" da v12, seção 34.)
- Correção (v15 = v14 + isto), sem mudar o módulo nem gastar espaço: o lmod confere `MOD2+$2FC` ($66FC-$66FF, dentro da área que o OS estraga) em vez de `MOD2+$3FC`, e o lm_mg guarda módulo[$2FC:$300] (`048d557f`; o da v11 seria `e5ad557f`: continua distinguindo módulos de versões diferentes). Os COVERS.BIN da v10/v13 valem.
- `teste_ligar_v15.py`: módulo da sessão anterior na WRAM + $6400-$66FF estragados, carga de 3 capas: v14 roda o código estragado (erro no emulador = trava no console); v15 relê os setores 257-258 e carrega. TUDO OK. `teste_capas_v10_emu` (agora confere a marca no endereço que o lmod usa) com a v15: TUDO OK. Navegação (v14): ok.
- `poc_capa_grande_v15/completo_817_capas/EDFC`: OS `caa12ed7…` + o COVERS.BIN da firmware_gerada. Hardware: **funcionou** (30/10/2026).

39. v15 na ferramenta = VERSÃO ESTÁVEL (30/10/2026)

- A ferramenta gera a v15 (`firmwares/OS_v15.BIN`, `caa12ed7…`; o módulo continua o MOD_v10). `compilar_exe.py` e o `conv_capa_grande.fonte0` (no .exe) usam o OS_v15. Teste da ferramenta: TUDO OK; o .exe recompilado gera o COVERS.BIN idêntico (0e495260…) e abre a janela.
- Backup estável: `BACKUP_ESTAVEL_v15_ferramenta_20261030/` (ferramentas sem caches, firmware v15 + 817 capas, o .zip do .exe, README + docs). Substitui o `BACKUP_ESTAVEL_v10_ferramenta_20261030` (que tem o travamento ao ligar).